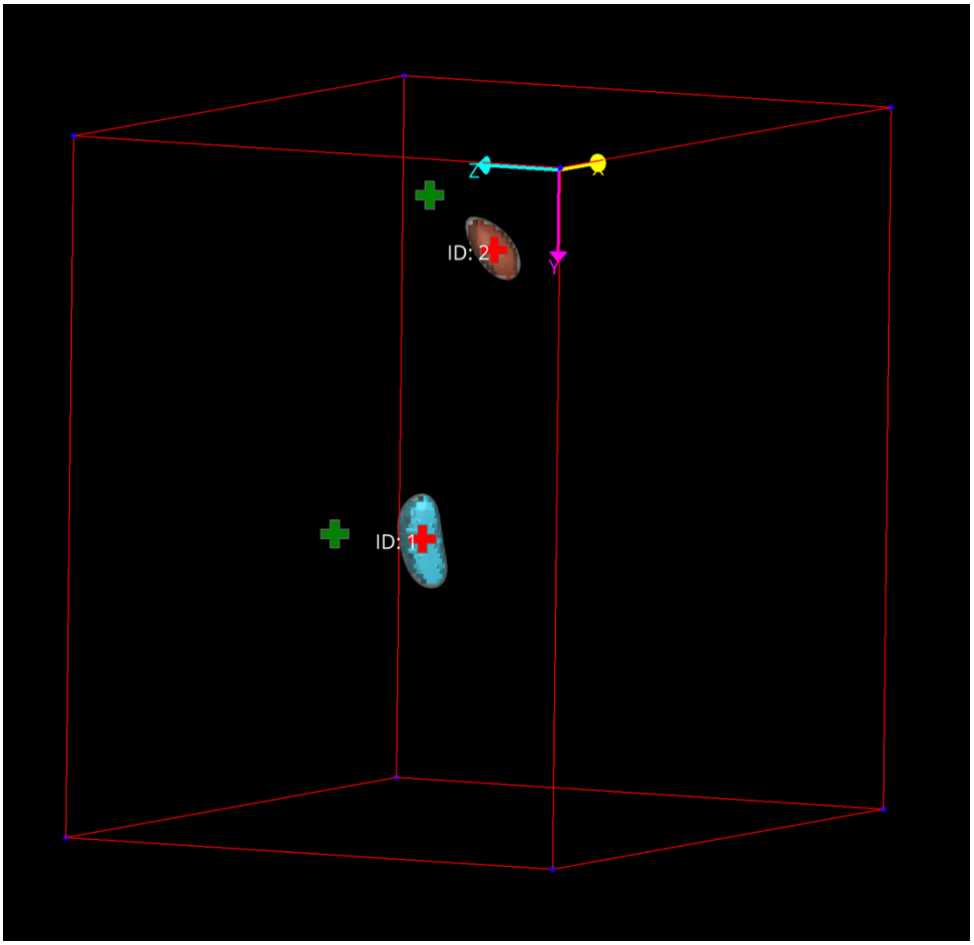
CentraleSupélec

lemta
Laboratoire Énergies & Mécanique Théorique et Appliquée

Projet de fin d'études

HOLOSOLVER

Reconstruction et suivi de bactéries dans l'espace tridimensionnel via une architecture NeRF hybride



09 avril 2026

Erwan GOURIOU

Louis BONNECAZE

Encadrants :

Simon BECKER

Jérémy FIX

Liudmyla KLOCHKO

Nicolas LOUVET

Table des matières

1 - Introduction *3*

1.1- L'holographie 3

1.2 - Fonctionnement des modèles NeRF 5

1.3 - Le lien entre l'holographie et les nerfs avec MorpHoloNet 6

1.4 - Présentation du projet 7

2 – L'implémentation de HoloSolver *8*

2.1 – Architecture du modèle HoloSolver 8

2.2 - Régularisations et Dynamique d'Optimisation 10

2.3 - Stratégie de Pré-entraînement et Prior Géométrique 12

2.4 - Ingénierie logicielle et Optimisation des performances 13

2.5 - Passage à l'échelle et Gestion de la VRAM (Chunking & Checkpointing) 15

2.6 - Générateur d'hologrammes : 16

3 – Résultats de nos tests *20*

3.1 - Validation de l'implémentation et Benchmarking (Cibles uniques) 20

3.2 - Validation sur cibles uniques : 27

3.3 - Passage à de multiples cibles 31

3.4 - Validation sur un hologramme réel 37

4 – Discussion et Conclusion *40*

4.1 - Interprétation des performances et limites du modèle : 40

4.2 -Difficultés rencontrés : 41

4.3 - Perspectives futures : 41

4.4 - Conclusion 42

1 - Introduction

1.1- L'holographie

L'holographie est au cœur du projet sur lequel nous travaillons. Dans un premier temps, nous allons en décrire ses principes généraux.

Il s'agit d'une technique d'imagerie permettant de reconstruire l'information 3D d'un objet à partir de l'enregistrement d'une onde lumineuse. Contrairement à une image classique 2D, elle capture à la fois l'intensité et la phase de la lumière. Le principe repose sur l'interférence entre une onde de référence qui est connu, et une onde diffractée par l'objet. L'enregistrement de cette figure d'interférence forme un hologramme.

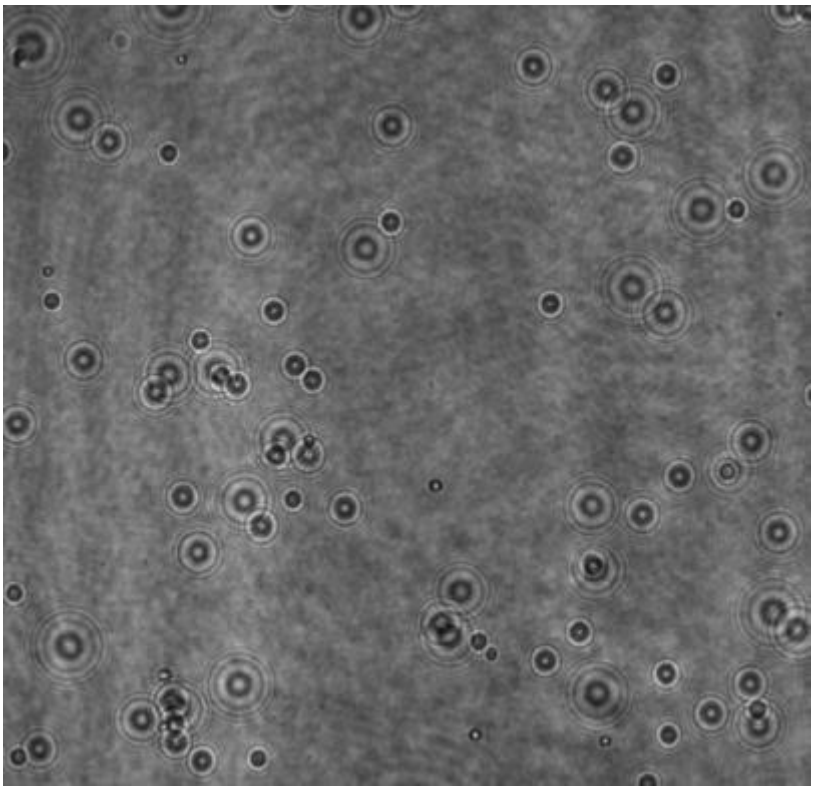


Image holographique brute d'un milieu particulaire

Avec l'évolution des capteurs et des capacités de calcul, l'holographie est aujourd'hui majoritairement numérique. L'acquisition des hologrammes se fait à l'aide d'une caméra puis la reconstruction de l'information est réalisée par calcul grâce à des à des méthodes de rétropropagation numérique, comme la méthode du spectre angulaire.

Le montage de Gabor est l'un des modèles les plus simples pour obtenir rapidement des hologrammes. La lumière traverse directement l'échantillon et l'hologramme est enregistré sans séparation physique entre l'onde de référence et l'onde diffractée par l'objet. Cette configuration simple permet d'acquérir rapidement des hologrammes, tout en conservant l'information

nécessaire à la reconstruction du champ lumineux. Grâce à un traitement numérique, on reconstruit ensuite l'image de l'objet à différentes profondeurs.

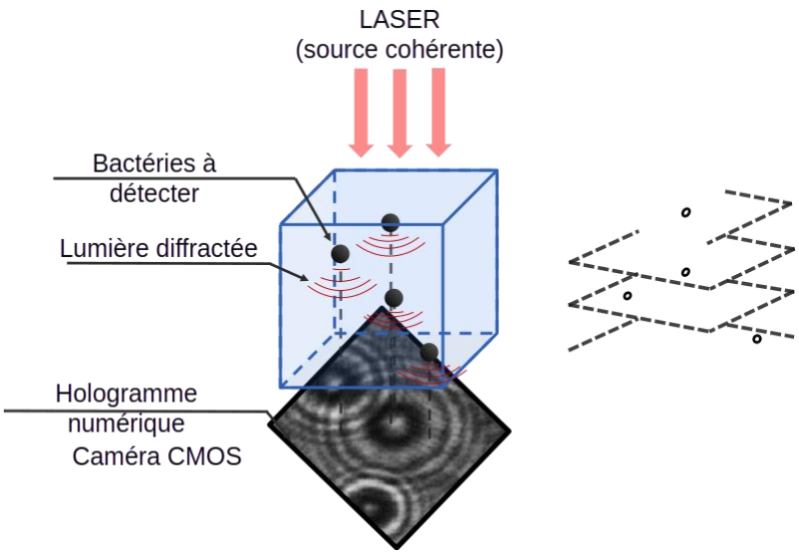
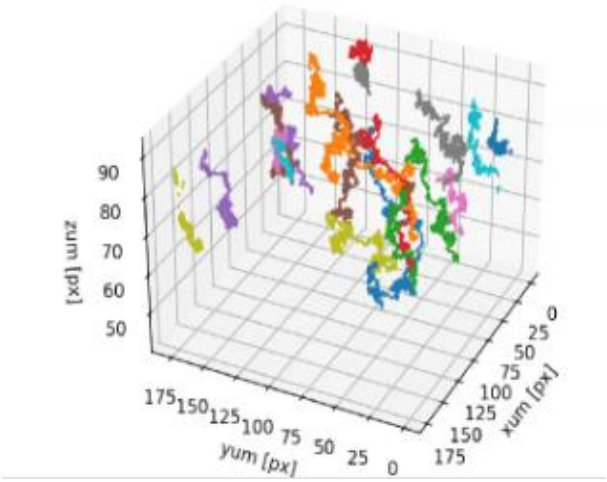


Schéma du montage de Gabor

À partir de l'hologramme 2D pris par la caméra, le traitement numérique des hologrammes suit plusieurs étapes successives. Un prétraitement est d'abord réalisé, comprenant la soustraction du fond et un filtrage fréquentiel afin de réduire le bruit. La reconstruction du champ lumineux est ensuite effectuée par rétropropagation numérique (à l'aide de la méthode du spectre angulaire, qui permet de calculer l'image à différentes profondeurs). Une fois le volume reconstruit, les objets sont détectés en identifiant les zones nettes à l'aide d'une fonction de focus, puis segmentés afin de déterminer leurs positions (barycentres).

En ce qui concerne les applications de l'holographie. Cette dernière est particulièrement adaptée à l'étude de systèmes complexes, notamment en microscopie et en mécanique des fluides. Elle permet le tracking de particules ou de bactéries. Elle permet également un accès à certaines grandeurs physiques liées à l'indice optique, comme la température ou la concentration. Cependant cette technique reste très sensible au bruit.



Tracking de bactéries grâce à l'holographie

1.2 - Fonctionnement des modèles NeRF

Les NeRF (Neural Radiance Fields) sont des modèles de deep learning utilisés en Computer Vision afin de reconstruire des scènes 3D à partir d'un ensemble d'images 2D. Ce modèle a été introduit en 2020 par l'équipe de Ben Mildenhall. Ils reposent sur l'apprentissage d'une représentation volumique continue d'une scène.

Le principe fondamental d'un NeRF est de modéliser une scène comme une fonction continue apprise par un réseau de neurones. Plus précisément, un réseau de type MLP (Multi-Layer Perceptron) prend en entrée :

- une position 3D dans l'espace (x, y, z),
- une direction d'observation (θ, ϕ),

et retourne :

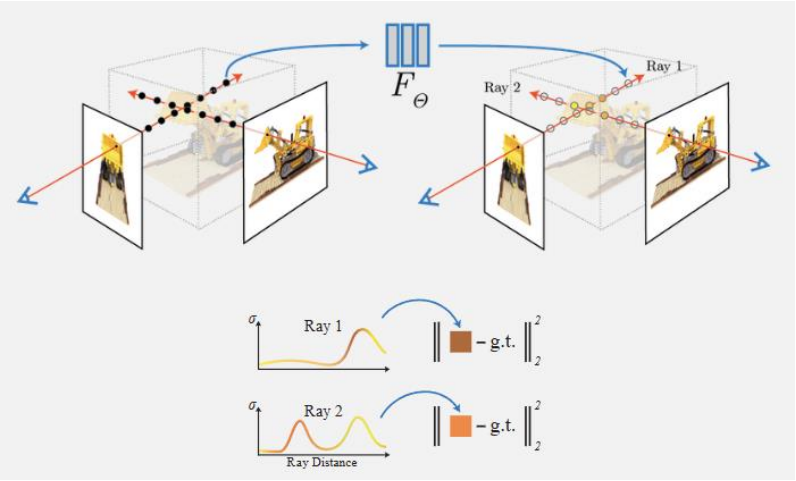
- une densité volumique (probabilité qu'un rayon soit absorbé à cet endroit)
- une couleur (RGB), dépendant du point et de la direction.

Ainsi, au lieu de représenter explicitement la géométrie (maillage, nuage de points), les modèles NeRF encode directement la scène dans les poids du réseau.

L'image est reconstruite en envoyant un rayon depuis la caméra pour chaque pixel. Ce rayon traverse la scène et on regarde ensuite les points le long de son trajet. Pour chacun de ces points, le modèle NeRF prédit une couleur et une densité. La couleur finale du pixel est ensuite obtenue en combinant les contributions de tous ces points. Les points les plus proches et les plus denses influencent davantage le résultat que les points plus éloignés ou transparents.

Le réseau est entraîné de manière supervisée à partir d'un ensemble d'images d'une même scène prises sous différents angles, avec les paramètres de caméra connus. L'objectif est de minimiser l'erreur entre les images réelles et celles synthétisées par le modèle. Ce processus permet au réseau d'apprendre implicitement :

- la géométrie de la scène,
- les propriétés d'apparence (textures, effets de vue),
- les phénomènes dépendant de la direction (réflexions, transparence).



Reconstitution de volume par NeRF

Les modèles NeRF sont aujourd'hui utilisés principalement dans le domaine du graphisme 3D et de la réalité virtuelle, où ils permettent de reconstruire des scènes très réalistes à partir d'images. Nous avons pu également voir qu'ils étaient utilisés dans la numérisation du patrimoine, notamment pour les restaurations… Beaucoup de projets les utilisant sont encore en développement mais leur utilisation est prometteuse et notamment avec des applications scientifiques comme en holographie comme nous allons le montrer juste après.

1.3 - Le lien entre l'holographie et les nerfs avec MorpHoloNet

Lors de notre étude de l'état de l'art, nos recherches se sont arrêtées sur une publication scientifique récente menée par une équipe de chercheurs coréens : « *Single-shot reconstruction of three-dimensional morphology of biological cells in digital holographic microscopy using a physics-driven neural network* ».

Leur article introduit **MorpHoloNet**, un modèle d'apprentissage profond hybride reposant sur les champs de radiance (similaires aux NeRF) couplés à un solveur physique. L'innovation majeure de ces chercheurs réside dans la capacité de leur modèle à reconstruire la morphologie 3D complète d'une cellule biologique à partir d'un unique hologramme 2D (*single-shot*). En intégrant les lois physiques de la diffraction optique directement au sein du réseau de neurones, le modèle s'optimise de manière auto-supervisée, parvenant à recréer le volume de la cellule sans nécessiter de vastes bases de données d'entraînement préalables.

Bien que cette architecture représente une avancée notable en imagerie optique, son champ d'application expérimental restait très restreint. En effet, les travaux de l'équipe coréenne se sont strictement limités à la reconstruction 3D statique d'une cible unique et isolée (une seule bactérie ou un seul globule rouge par image). Leur étude ne visait ni l'analyse d'environnements

denses (cibles multiples), ni la problématique de la localisation et du suivi spatial (*tracking*), qui constituent pourtant les véritables enjeux de l'analyse biologique moderne.

Cette limite applicative a constitué le point de départ idéal pour notre projet. Nous avons postulé que la capacité intrinsèque de l'approche NeRF à cartographier finement la densité de la matière en 3D pouvait être étendue pour résoudre notre problématique de *tracking*. Au-delà de la simple modélisation d'une forme, l'adaptation de cette architecture mathématique devait nous permettre de détecter et de positionner spatialement plusieurs bactéries au sein d'un même volume d'étude. Associer la rigueur mathématique du *tracking* 3D à la visualisation morphologique détaillée de chaque cible représentait un axe de développement extrêmement prometteur, donnant ainsi l'impulsion initiale pour la conception de notre propre modèle.

1.4 - Présentation du projet

Présentation du LEMTA :

LEMTA est un laboratoire de recherche du CNRS et de l'Université de Lorraine. Ce dernier est situé à Nancy. Il regroupe près de 200 personnes principalement des chercheurs, des ingénieurs, et des doctorants et mène des travaux principalement autour de la mécanique, des fluides, et des phénomènes énergétiques.

Dans le cadre de leurs recherches, Simon Becker et Nicolas Louvet, cherchent à suivre et analyser la dynamique de colonisation de bactéries en 3D sur une surface. Pour cela, ils utilisent un dispositif d'holographie numérique qui permet de reconstruire la position des bactéries à partir de motifs de diffraction enregistrés par une caméra. Cependant, le traitement actuel présente plusieurs limites :

- Il est complexe et repose sur énormément de paramètres
- Il nécessite une expertise importante pour fonctionner correctement
- Les paramètres sont souvent globaux, ce qui limite la précision selon les situations (orientation, position des particules, etc.)

Il est ainsi difficile d'adapter localement le traitement pour améliorer la détection. Leur besoin principal est donc de rendre ce traitement plus robuste, automatisé et adaptable, en réduisant la dépendance aux réglages manuels.

Au vu de l'introduction réalisée précédemment (présentation des NeRF et lien avec l'holographie, notamment avec MorpHoloNet), nous nous sommes appuyés sur ces modèles pour améliorer la localisation des bactéries à partir des hologrammes. Une partie du travail a également porté sur la reconstruction 3D des bactéries, car il s'agit d'un aspect important des NeRF et il aurait été dommage de ne se limiter qu'au tracking.

2 – L'implémentation de HoloSolver

2.1 – Architecture du modèle HoloSolver

Notre pipeline de reconstruction, baptisé HoloSolver, repose sur une architecture hybride de type *Physics-Informed Neural Network* (PINN). Il couple un réseau de neurones (représentation continue de l'espace) avec un solveur physique différentiable (lois de l'optique ondulatoire). Le modèle se divise ainsi en deux composantes fondamentales : le réseau MorpHoloNet pour l'inférence de la géométrie 3D, et le Modèle Physique pour la simulation de la propagation lumineuse.

2.1.1 - Inférence spatiale et encodage positionnel (MorpHoloNet)

L'objectif du réseau MorpHoloNet est d'apprendre une fonction continue f mapping une coordonnée spatiale 3D vers une densité optique locale :

$$\boldsymbol{f} : (\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{z}) \rightarrow \boldsymbol{\sigma}$$

où $\sigma \in [0, 1]$. Pour borner strictement cette densité, la couche de sortie du réseau utilise une fonction d'activation Sigmoïde. L'architecture interne est volontairement minimaliste (un Perceptron Multicouche à 3 couches cachées), la complexité du problème résidant davantage dans l'encodage des entrées que dans la profondeur du réseau.

Les réseaux de neurones classiques peinant à modéliser les variations à haute fréquence (un phénomène connu sous le nom de *Spectral Bias*), nous appliquons un encodage positionnel basé sur les caractéristiques de Fourier (*Fourier Features*).

Au cours de nos expérimentations, nous avons implémenté des méthodes d'encodage par grille de hachage (*Hash Encoding* de type Instant-NGP) dans le but d'accélérer l'apprentissage. Toutefois, ces méthodes généraient un bruit haute fréquence et des micro-discontinuités spatiales (brouillard) incompatibles avec la continuité requise par les équations de propagation d'ondes. Nous avons donc privilégié un encodage positionnel paramétrique classique.

La particularité de notre implémentation réside dans le **découplage anisotrope** de l'encodage. Les axes transverses (x, y) et l'axe de profondeur z sont encodés par deux projections distinctes :

$$\boldsymbol{\gamma}_{xy}(\boldsymbol{x}, \boldsymbol{y}) = \left[\mathbf{sin}\left(2\boldsymbol{\pi}\mathbf{B}_{xy}\boldsymbol{v}_{xy}\right), \mathbf{cos}\left(2\boldsymbol{\pi}\mathbf{B}_{xy}\boldsymbol{v}_{xy}\right)\right]$$

$$\boldsymbol{\gamma}_{z}(\boldsymbol{z}) = \left[\mathbf{sin}(2\boldsymbol{\pi}\mathbf{B}_{z}\boldsymbol{z}), \mathbf{cos}(2\boldsymbol{\pi}\mathbf{B}_{z}\boldsymbol{z})\right]$$

où $v_{xy} = [x, y]^T$ représente le vecteur des coordonnées transverses et z la coordonnée de profondeur. Les termes B_{xy} et B_z correspondent aux matrices de projection générant les caractéristiques de Fourier.

Afin de permettre au réseau d'apprendre les variations structurelles de différentes échelles, les poids de ces matrices ne sont pas appris, mais initialisés de manière statique selon une distribution normale tronquée $\mathcal{N}(0, \sigma^2)$, où σ est un hyperparamètre d'échelle (`gaussian_scale`) contrôlant la bande passante des fréquences encodées.

Ce découplage est physiquement impératif. Dans notre configuration, les échelles spatiales diffèrent fondamentalement : les dimensions x et y sont liées à la taille discrète des pixels du capteur, tandis que z représente une dimension continue en micromètres. Un encodage couplé aurait provoqué une déformation de l'espace des fréquences et un étirement aberrant de la morphologie des bactéries sur l'axe Z.

2.1.2 - Modèle Physique : Propagation par Spectre Angulaire

La seconde composante de l'architecture simule le parcours de la lumière à travers le volume 3D prédit, couche par couche, du plan le plus éloigné (z_{max}) jusqu'au capteur ($z = 0$).

À chaque profondeur z, la densité $\sigma(x, y, z)$ prédite par le NeRF est convertie en un retard de phase local. Le champ complexe incident U_{in} traversant cette tranche subit une modulation de phase pour produire le champ sortant U_{out} :

$$\boldsymbol{U_{out}(x, y) = U_{in}(x, y)\exp\left(i \cdot \Delta\phi \cdot \sigma(x, y, z)\right)}$$

où $\Delta\phi$ est le déphasage maximal autorisé, appris dynamiquement par le modèle en tant que paramètre optimisable (`phase_shift`).

La propagation de ce champ complexe d'une tranche à la suivante (sur une distance Δ_z) est calculée de manière rigoureuse via la méthode du **Spectre Angulaire** (*Angular Spectrum Method*). En passant dans le domaine fréquentiel via une transformée de Fourier 2D ($\mathcal{F}$), on multiplie le spectre par la fonction de transfert de l'espace libre G, puis on applique la transformée inverse :

$$\boldsymbol{U_{prop} = \mathcal{F}^{-1}\left\{\mathcal{F}\{U_{out}\} \cdot \exp\left(ik\Delta_z\sqrt{1 - (\lambda f_x)^2 - (\lambda f_y)^2}\right)\right\}}$$

Cette implémentation exploite pleinement le calcul tensoriel complexe de PyTorch (`torch.fft.fft2`), permettant la rétropropagation du gradient analytique à travers les lois de l'optique.

2.1.3 - Fonction de Coût

Une fois le champ propagé jusqu'au plan du capteur ($z = 0$), nous calculons son intensité théorique $I_{pred} = |U_{z=0}|^2$.

La supervision du modèle est alors effectuée en comparant cette intensité à l'hologramme réel capturé I_{target}.

Nos études empiriques sur ce problème inverse mal posé ont révélé qu'une erreur absolue moyenne (Loss L1) entraînait une forte instabilité de l'optimisation (évanouissement du gradient menant à une reconstruction optique nulle, ou "image noire"). Nous avons par conséquent adopté la norme L2 (Erreur Quadratique Moyenne - MSE), dont la courbure garantit une convergence plus douce et pénalise de manière quadratique les franges d'interférence mal positionnées :

$$\mathcal{L}_{phys} = \frac{1}{N}\sum\left(I_{target} - I_{pred} \right)^2$$

2.1.4 - Paramétrage dynamique du système physique

Au-delà de l'inférence spatiale, la classe globale `HoloSolver` agit comme un orchestrateur optimisant simultanément le réseau de neurones et les paramètres du système optique. En effet, des variables physiques cruciales, telles que le déphasage maximal (`phase_shift`) et l'intensité de la lumière incidente (`incident_light`), sont déclarées comme des paramètres apprenables de l'optimiseur (`nn.Parameter`).

2.2 - Régularisations et Dynamique d'Optimisation

Le problème inverse en holographie (retrouver un volume 3D à partir d'une projection 2D d'intensité) est fondamentalement mal posé. Il existe une infinité de répartitions de matière capables de générer un motif d'interférence similaire. Pour guider le réseau de neurones vers la solution physiquement réaliste, une ingénierie minutieuse de la fonction de perte et des hyperparamètres a été nécessaire.

2.2.1 - Conditions aux limites (Boundary Conditions)

Sans contrainte spatiale, le modèle a tendance à exploiter les degrés de liberté du volume en plaçant de la matière sur les frontières de l'espace étudié. Ce phénomène s'explique par la nature de la propagation d'ondes : le réseau crée des artefacts sur les bords pour compenser l'information manquante hors du champ de vision. Pour pallier cela, nous avons implémenté une pénalisation stricte sur les 6 faces limitrophes du volume $\partial\Omega$. Cette régularisation par conditions aux limites (BC) utilise une norme L2 :

$$\mathcal{L}_{BC} = \frac{1}{N_{\partial\Omega}} \sum_{(x,y,z)\in\partial\Omega} (\sigma(x,y,z))^2$$

Appliquée à chaque itération, elle force la densité σ à tendre vers zéro sur les bords, concentrant ainsi la matière de la bactérie au centre du volume sans dégrader la reconstruction de l'hologramme.

2.2.2 - Régularisations exploratoires et Apprentissage par Curriculum

Durant la conception du modèle, d'autres termes de régularisation classiques en vision par ordinateur ont été expérimentés, notamment la pénalisation de la variation totale (TV Loss) et la parcimonie (Sparsity L1). Bien qu'utiles pour atténuer le bruit haute fréquence généré par les approches de type *Hash Encoding*, ces régularisations se sont avérées destructrices avec notre encodage positionnel. En effet, un poids de *sparsity* trop élevé forçait le modèle à converger vers une solution triviale vide (image noire), provoquant un évanouissement des gradients (*gradient vanishing*).

Dans une optique d'accélération, nous avons également testé une approche de *Curriculum Learning*. La cible (l'hologramme réel) était initialement floutée par un noyau Gaussien de variance , puis progressivement rendue nette sur 1000 époques, accompagnée d'un ordonnanceur de taux d'apprentissage (*LR Scheduler*). L'objectif était de lisser la fonction de perte pour guider l'optimiseur. Toutefois, en holographie, l'information de phase est contenue dans les franges d'interférence à très haute fréquence : le floutage détruisait cette information vitale, rendant le *scheduler* et le *blurring* inefficaces, voire contre-productifs. Nous les avons donc désactivés dans la configuration finale.

2.2.3 - Calibrage des Hyperparamètres et de l'Optimiseur Adam

Les gains de performance majeurs ont finalement été obtenus par un réglage fin de la dynamique d'optimisation et des hyperparamètres du NeRF :

Découplage des taux d'apprentissage :

L'optimisation simultanée d'un réseau de neurones complexe (milliers de poids) et de scalaires physiques (2 paramètres : `phase_shift` et `incident_light`) nécessite des échelles de gradient différentes. Nous avons donc scindé le taux d'apprentissage : un *Learning Rate* prudent de 5×10^{-4} pour le NeRF, et un taux plus agressif de 5×10^{-3} pour l'ajustement rapide de la physique.

Inertie de l'optimiseur (Beta 2) :

L'analyse des courbes d'apprentissage a révélé de longs plateaux de perte (stagnation entre 2000 et 4000 époques). Pour résoudre cela, le paramètre β_2 de l'optimiseur Adam a été réduit de 0.999 (standard) à 0.95. Mathématiquement, cela réduit la "mémoire" (moyenne mobile exponentielle) des gradients passés, permettant à l'optimiseur de changer de direction plus rapidement et de s'échapper des zones de selle (*saddle points*).

Bande passante de l'encodage (*Gaussian Scale*) :

En augmentant le paramètre `gaussian_scale` à 10, nous avons élargi le spectre des fréquences spatiales encodées. Le réseau est ainsi devenu capable de modéliser des détails morphologiques beaucoup plus fins dès les premières itérations.

L'abandon des régularisations instables au profit des conditions aux limites, couplé à ce calibrage minutieux de l'optimiseur, a drastiquement réduit la complexité temporelle. Pour la reconstruction d'une cible unique, le temps de convergence est passé d'une fourchette de 5000-9000 époques à seulement **1000-3000 époques** (selon la complexité du motif de diffraction), assurant un gain de performance indispensable pour la modélisation ultérieure de bactéries multiples.

2.3 - Stratégie de Pré-entraînement et Prior Géométrique

Bien que le calibrage fin de nos hyperparamètres ait drastiquement réduit le temps de convergence, la résolution stricte du problème inverse par la seule simulation de la physique ondulatoire reste hautement sensible aux minima locaux. Pour garantir la robustesse du modèle et s'affranchir définitivement des ambiguïtés de profondeur inhérente à l'holographie in-line, nous avons introduit une phase de pré-entraînement agissant comme un **Prior géométrique**.

L'objectif de cette phase n'est pas de résoudre les micro-détails optiques, mais d'initialiser les poids du NeRF pour qu'ils prédisent une distribution de matière cohérente avant même l'activation du modèle physique. Sachant que notre pipeline de données fournit la vérité terrain des coordonnées spatiales (x_0, y_0, z_0) et le rayon physique (r) des bactéries cibles, nous forçons le réseau à apprendre une approximation de ces formes pendant les **1500 premières époques.**

2.3.1 - Modélisation par distribution Gaussienne Multivariée (MVN)

Pour modéliser mathématiquement une cible continue dans l'espace 3D, nous utilisons une distribution Gaussienne Multivariée dont l'intensité décroît depuis le centre de la bactérie. La cible locale pour un voxel donné de coordonnées normalisées (x, y, z) est définie par :

$$\boldsymbol{G_{local}(x, y, z) = \exp\left(-0.5\left[\frac{(x - x_0)^2}{\sigma_x^2} + \frac{(y - y_0^2)}{\sigma_y^2} + \frac{(z - z_0^2)}{\sigma_z^2}\right]\right)}$$

Un choix de conception crucial a été d'initialiser des **ellipsoïdes aplatis** plutôt que des sphères parfaites. Dans notre code, la variance sur l'axe de profondeur (σ_z^2) intègre un facteur de compression de 0.5. Ce choix répond à une contrainte physique bien connue en holographie in-line : la faible résolution axiale. En raison de la perte de la phase optique lors de la capture, les algorithmes de reconstruction ont tendance à "étaler" la matière le long de l'axe optique, transformant des objets sphériques en formes étirées (un artefact d'élongation axiale). En

initialisant notre Prior géométrique avec une forme artificiellement compressée en Z, nous anticipons et contrebalançons cet étirement naturel provoqué par le solveur physique.

2.3.2 - Union spatiale Multi-Cibles et Limites d'Activation

Le passage de l'entraînement d'une cible unique à un environnement multi-bactéries soulève un défi d'intersection spatiale. Si deux ellipsoïdes cibles sont proches, la somme algébrique de leurs distributions Gaussiennes créerait des zones où la densité cible dépasserait 1.0. Or, la sortie de notre NeRF est strictement bornée dans l'intervalle $[0, 1]$ par une fonction Sigmoïde. Pour préserver la viabilité mathématique des gradients, nous remplaçons la somme par une **union topologique** via l'opérateur maximum :

$$\boldsymbol{G_{global}} = \max_{\boldsymbol{i \in cibles}} \left(\boldsymbol{G_i(x, y, z)} \right)$$

Cette opération garantit que la densité cible globale n'excédera jamais, évitant ainsi la saturation de la Sigmoïde et l'explosion des gradients.

2.3.3 - Transition d'optimisation et préservation du Prior

Le passage de la phase de pré-entraînement à l'entraînement du modèle induit un changement radical du paysage d'erreur. Expérimentalement, cette transition brutale entraînait la destruction immédiate de la connaissance à priori acquise, un phénomène lié à la mécanique interne de l'optimiseur Adam. En appliquant l'historique d'inertie (le *momentum*) des gradients du pré-entraînement à la nouvelle perte d'entraînement du modèle, l'optimiseur générait une instabilité majeure conduisant à l'explosion des gradients.

Pour pallier ce problème, nous avons implémenté une réinitialisation stricte de l'optimiseur à la transition exacte entre ces deux phases. En purgeant l'état interne d'Adam (momentum et variance) et en réduisant simultanément le taux d'apprentissage de manière drastique (pour neutraliser la violente correction de biais inhérente au redémarrage de l'algorithme), l'optimiseur aborde la phase physique avec une vélocité vierge. Cette stratégie permet donc de préserver le *Prior* morphologique et positionnel lors du passage à la phase d'entraînement du modèle.

2.4 - Ingénierie logicielle et Optimisation des performances

La simulation de la propagation lumineuse par spectre angulaire, couplée à l'inférence continue d'un champ de radiance, représente une charge computationnelle extrême. Lors de nos premières implémentations, la reconstruction d'une cible complexe (ex: *Isotonic RBC*, impliquant 200 plans de profondeur $z_{max} = 100$ et $\Delta z = 0.5$) nécessitait environ 50 minutes pour 10 000 époques sur un GPU NVIDIA RTX 3090 (24 Go de VRAM). La vitesse d'entraînement plafonnait à **3,7 itérations par seconde (it/s)**.

Afin de réduire ce goulot d'étranglement (*bottleneck*) et de fluidifier la phase d'expérimentation, nous avons opéré une refonte de l'architecture logicielle autour de trois axes d'optimisation majeurs.

2.4.1 - Vectorisation spatiale des inférences (Batched Query)

Initialement, la prédiction de la densité s'effectuait plan par plan (boucle `for` sur l'axe Z), ce qui sous-utilisait les cœurs CUDA du GPU en raison de la petite taille des tenseurs manipulés. Nous avons restructuré le pipeline pour pré-calculer une grille tridimensionnelle complète (`self.coords_3d`). Au lieu d'appeler le réseau 200 fois par itération, le NeRF traite désormais l'intégralité du volume en une seule passe *forward*.

Cette vectorisation stricte sature efficacement les unités de calcul du GPU et a permis un gain immédiat de **+1,3 it/s**. Ce gain temporel s'accompagne néanmoins d'un compromis matériel : traiter tout le volume simultanément nécessite de stocker l'ensemble des activations intermédiaires pour la rétropropagation (*backward pass*), ce qui augmente drastiquement l'empreinte mémoire (VRAM).

2.4.2 - Mise en cache VRAM et Pré-calculs de la Physique

La méthode du spectre angulaire requiert la génération d'une grille de fréquences spatiales (`u`, `v`) et d'un noyau quadratique (`S2`) à chaque étape de propagation. Re-calculer ces matrices constantes via le CPU à chaque époque générait une latence de transfert mémoire CPU vers GPU catastrophique. Nous avons déporté l'ensemble de ces constantes directes dans la mémoire statique de la carte graphique lors de l'initialisation de la classe `PhysicsModel` en utilisant les *Buffers* de PyTorch.

La même logique a été appliquée aux tenseurs utilitaires dans `HoloSolver` (`ones_grid`, `zeros_grid`, tenseurs complexes). Cette suppression totale des transferts PCIe redondants et des réallocations dynamiques a constitué l'optimisation la plus rentable de notre étude, offrant un bond de **+4,0 it/s**.

2.4.3 - Accélération matérielle et Compilation JIT

Pour achever cette optimisation et pallier la hausse de consommation mémoire induite par la vectorisation, nous avons exploité la flexibilité matérielle des *Tensor Cores* (architecture NVIDIA Ampere) et logicielle de PyTorch 2.0 :

Précision Mixte Automatique (AMP) :

En encapsulant la passe *forward* avec `torch.autocast`, PyTorch bascule dynamiquement les opérations compatibles (comme les couches du NeRF) au format `bfloat16` (ou `float16` sécurisé par un `GradScaler`). Ce passage en 16 bits divise par deux l'empreinte VRAM du réseau de neurones.

Format TF32 pour la physique :

Les opérations non compatibles avec le 16 bits (nombres complexes, optique ondulatoire) restent automatiquement en 32 bits pour garantir la stabilité numérique. Pour les accélérer, l'instruction `torch.set_float32_matmul_precision('high')` active le format matériel TensorFloat-32 (TF32), qui effectue les multiplications matricielles à la vitesse du 16 bits tout en conservant la plage dynamique du 32 bits.

Compilation Dynamo :

L'instruction `torch.compile(model.Nerf)` applique une compilation *Just-In-Time* (JIT). Le graphe d'exécution est analysé en amont pour fusionner les opérations (*Operator Fusion*), ce qui réduit considérablement la surcharge de l'interpréteur Python.

Bilan des optimisations :

La synergie de ces trois niveaux d'optimisation (algorithmique, mémoire, et matérielle) a transformé les performances du modèle. Pour la cible de référence, la vitesse de traitement est passée de **3,7 it/s à 9,0 it/s**. Le temps nécessaire pour franchir les 10 000 époques a été réduit de 50 minutes à seulement **20 minutes**, ce qui représente une augmentation globale des performances de **243 %**.

2.5 - Passage à l'échelle et Gestion de la VRAM (Chunking & Checkpointing)

Bien que notre vectorisation spatiale (décrite en Section 2.5.1) ait maximisé la vitesse de traitement (it/s), elle a engendré un coût indirect : une consommation de VRAM extrêmement élevée. En effet, la construction d'un graphe de calcul dynamique englobant à la fois les inférences d'un perceptron multicouche et la simulation optique par transformées de Fourier sur un volume entier requiert de stocker des millions d'activations intermédiaires.

Afin de rendre notre modèle résilient et capable de traiter des hologrammes haute résolution (512x512 pixels), il était impératif d'implémenter des stratégies de réduction d'empreinte mémoire pour éviter les erreurs de saturation du GPU (*Out of Memory* - OOM). Pour ce faire, deux mécanismes modulables ont été intégrés :

2.5.1 - Le traitement spatial par lots (Chunking) :

Plutôt que d'interroger le NeRF avec l'intégralité de la grille 3D simultanément (un tenseur de coordonnées massif), l'espace est découpé en sous-ensembles continus appelés *chunks* (par exemple, des lots de 10 000 000 de coordonnées). Le modèle traite ces lots séquentiellement lors de la passe *forward* et concatène les prédictions (`torch.cat`) pour reconstituer le volume 3D. Cette approche borne strictement la quantité de VRAM allouée lors de l'inférence spatiale, indépendamment de la taille de l'hologramme cible.

2.5.2 - Le Gradient Checkpointing (Re-calcul des activations) :

La propagation physique couche par couche (`angular_spectrum_propagator`) génère un graphe de calcul extrêmement profond, saturant rapidement la mémoire lors de la sauvegarde des gradients. Nous avons appliqué la fonction `torch.utils.checkpoint` à la fois sur le réseau de neurones et sur le propagateur physique. Mathématiquement, cette technique s'appuie sur un compromis temps/mémoire : elle ordonne à PyTorch de ne pas stocker les tenseurs d'activation intermédiaires lors de la passe *forward*. Ces tenseurs sont libérés de la VRAM, puis **recalculés à la volée** lors de la passe *backward* pour évaluer le gradient.

2.5.3 - Le compromis opérationnel :

L'activation conjointe du *Chunking* et du *Checkpointing* réduit drastiquement l'empreinte VRAM, rendant possible l'étude de très grands volumes. La contrepartie directe est une forte dégradation du nombre d'itérations par seconde (le *Checkpointing* obligeant le GPU à effectuer la passe *forward* deux fois). Néanmoins, ces hyperparamètres étant configurables par l'utilisateur, ils offrent au modèle `HoloSolver` une flexibilité indispensable pour s'adapter dynamiquement aux limites matérielles de toute machine cible.

2.6 - Générateur d'hologrammes :

Pour l'entraînement du modèle, nous avons besoin d'hologrammes. Toutefois, les hologrammes réels sont coûteux à produire et nécessitent une installation spécifique, uniquement disponible au LEMTA. Nous avons donc dû recourir à un simulateur d'hologrammes. Ce dernier étant parfaitement déterministe, nous en détaillerons le principe dans cette partie. Grâce à Simon Becker, qui nous a fourni son simulateur, nous avons pu l'adapter à notre modèle et l'utiliser pour générer des d'hologrammes, tout en faisant varier les paramètres. Cela nous a permis d'étudier le comportement de notre modèle avec plusieurs configurations.

2.6.1 - Modélisation des bactéries

Le simulateur modélise les bactéries comme des cylindres de longueur L et d'épaisseur D . Elles sont définies par leur position (x, y, z) et leurs angles d'orientation (θ, Φ) .

2.6.2 Modélisation Physique de Propagation

Le simulateur repose sur l'approximation scalaire de la diffraction et utilise la méthode de la propagation du spectre angulaire. On initialise d'abord un champ électrique incident E_0 :

$$\boldsymbol{E_0(x, y, 0) = \sqrt{I_{noise}(x, y)}.\, e^{-j\phi_0}}$$

Le volume est divisé en N_z tranches de profondeur Δ_z. Le champ se propage à travers le milieu d'indice n_{milieu} selon l'équation de Helmholtz. Dans le domaine fréquentiel, le champ au plan $z + \Delta_z$ est ensuite calculé par :

$$\boldsymbol{A(u, v, z + \Delta_z) = A(u, v, z).\, H(u, v, \Delta_z)}$$

Où $A(u,v)$ est la transformée de Fourier spatiale du champ et H est la fonction de transfert de l'espace libre :

$$\boldsymbol{H(u,v,\Delta_z) = e^{jn_{milieu}2\pi\sqrt{1-\left(\frac{\lambda u}{n_{milieu}}\right)^2-\left(\frac{\lambda v}{n_{milieu}}\right)^2}}}$$

Une fois la propagation effectuée dans le domaine fréquentiel, il est nécessaire de revenir dans l'espace réel (spatial) :

$$\boldsymbol{E(x,y,z+\Delta_z) = \mathrm{F}^{-1}\{A(u,v,z+\Delta_z)\}}$$

Lorsqu'un objet d'indice n_0 est présent dans une tranche, il induit un déphasage local $\Delta\phi$. Le champ traversant l'objet est modifié par :

$$\boldsymbol{E_{out}(x,y) = E_{in}(x,y).e^{-j\Delta\phi(x,y)}}$$

Le déphasage est proportionnel à la différence d'indice et à l'épaisseur traversée Δ_z :

$$\boldsymbol{\Delta\phi = \frac{2\pi\Delta_z(n_0 - n_{milieu})}{\lambda}}$$

2.6.3 - Paramètres physiques et implémentation

L'ensemble des paramètres physiques nécessaires à la génération des hologrammes est centralisé dans un fichier de configuration JSON. Ce fichier permet de définir notamment :

- **La source lumineuse :** Longueur d'onde λ et intensité moyenne.
- **Les objets :** Morphologie des bactéries (longueur L , épaisseur D), indice de réfraction…
- **Le milieu et le volume :** Indice du milieu, dimensions du volume de simulation et nombre de plans…
- **Le système optique :** Taille réelle des pixels du capteur, grossissement de l'objectif, distance de propagation…
- **Les perturbations :** Niveau de bruit gaussien ajouté pour simuler les conditions réelles d'acquisition.

Pour rappel, pour simuler numériquement la propagation, le volume physique doit être discrétisé en une grille de voxels. La taille d'un voxel dans le plan (x,y) notée Δ_{xy}, correspond à la résolution ramenée dans le plan de l'objet, calculée en divisant la taille du pixel physique sur le capteur de la caméra par le grossissement du système :

$$\boldsymbol{\Delta_{xy} = \frac{p_{camera}}{G}}$$

Ainsi, Le passage des coordonnées réelles (x_m, y_m, z_m) exprimées en mètres vers les indices de la matrice de calcul du volume discrétisé $(x_{vox}, y_{vox}, z_{vox})$se fait selon les relations suivantes :

$$\boldsymbol{x_{vox} = \frac{x_m}{\Delta_{xy}} \qquad y_{vox} = \frac{y_m}{\Delta_{xy}} \qquad z_{vox} = \frac{z_m}{\Delta_z}}$$

La résolution selon l'axe optique Δ_z, est définie par le pas de discrétisation choisi pour la propagation de plan en plan. Ce paramètre est configuré directement dans le fichier de configuration.

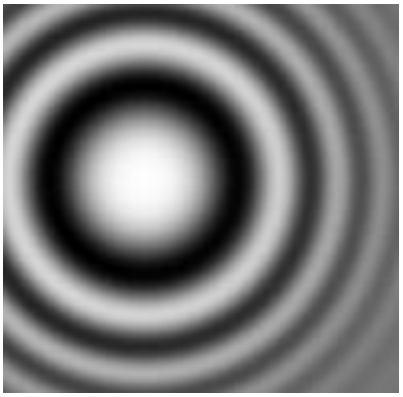
Exemple : Avec des pixels de 5.5μm et un grossissement de x40, chaque voxel représente latéralement 137.5nm. Si l'on prend ensuite une résolution de 1024x1024 pixels le simulateur modélise une fenêtre physique de $N\Delta_{xy}$ soit 140.8 μm . Si l'on utilise une résolution de 128x128 pixels, le simulateur ne modélise qu'une fenêtre physique de 17.6 µm. Il est donc plus intéressant pour nous d'utiliser des hologrammes avec une forte résolution pour accumuler plus d'informations mais il faut trouver un équilibre avec la mémoire disponible sur les GPU (VRAM) qui est contrainte.

2.6.4 - Génération

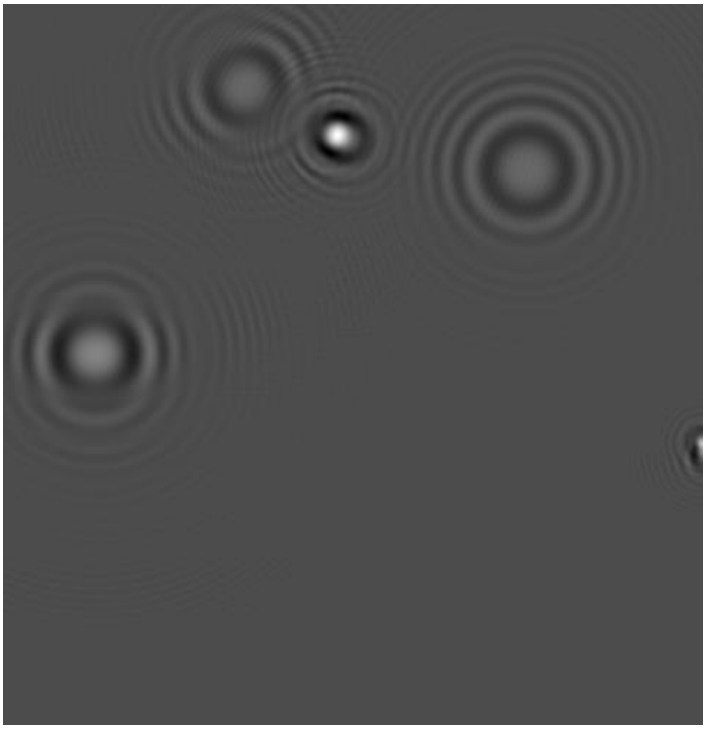
Une fois le volume initialisé, les bactéries sont placées de manière aléatoire sur la grille de voxel (leur nombre dépend des paramètres fixés en entrée). La lumière traverse ensuite ce volume par une boucle de propagation itérative, calculant le champ plan par plan de jusqu'au capteur comme expliqué précédemment. L'hologramme final est ainsi obtenu par le calcul de l'intensité du champ électrique au dernier plan :

$$\boldsymbol{I(x,y) = \left|E\left(x,y,z_{final}\right)\right|^2}$$

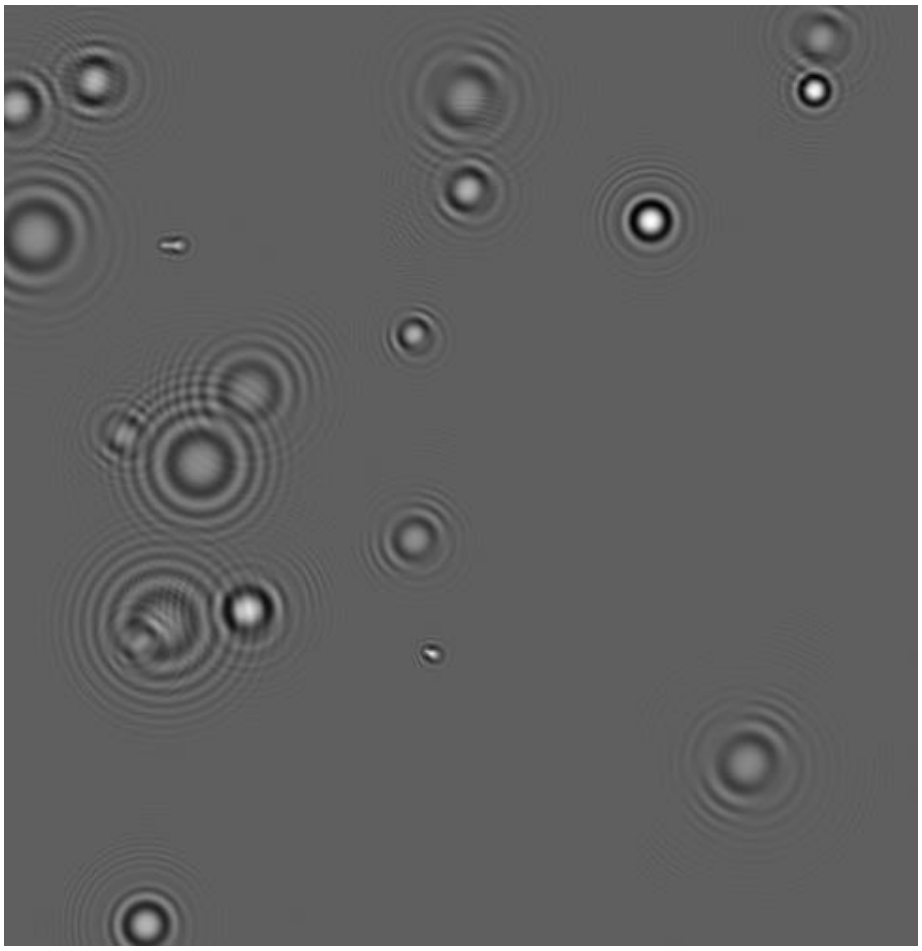
2.6.5 - Exemples d'hologrammes générés



Hologramme 128x128 – 1 bactérie



Hologramme 512x512 – 5 bactéries



Hologramme 1024x1024 – 20 bactéries

3 – Résultats de nos tests

3.1 - Validation de l'implémentation et Benchmarking (Cibles uniques)

L'objectif premier de cette section est de valider notre architecture `HoloSolver` en la confrontant directement au modèle de l'état de l'art dont elle s'inspire : `MorpHoloNet`. Cette comparaison vise à démontrer que notre implémentation PyTorch est non seulement fonctionnelle, mais qu'elle bat le modèle de référence sur trois axes majeurs : la fidélité de la reconstruction 3D, la précision de la localisation spatiale (*tracking*), et l'efficacité computationnelle (temps d'entraînement).

Afin de garantir une évaluation parfaitement équitable, les deux modèles ont été entraînés dans un environnement strictement contrôlé. Les expériences partagent les mêmes paramètres optiques et ont été exécutées sur le même matériel (GPU NVIDIA RTX 3090). Le jeu de données de référence, fourni par les auteurs originaux, est composé de quatre hologrammes expérimentaux : une bactérie (*E. coli*) et trois globules rouges dans différents états osmotiques (*Isotonic, Hypotonic, Hypertonic RBC*).

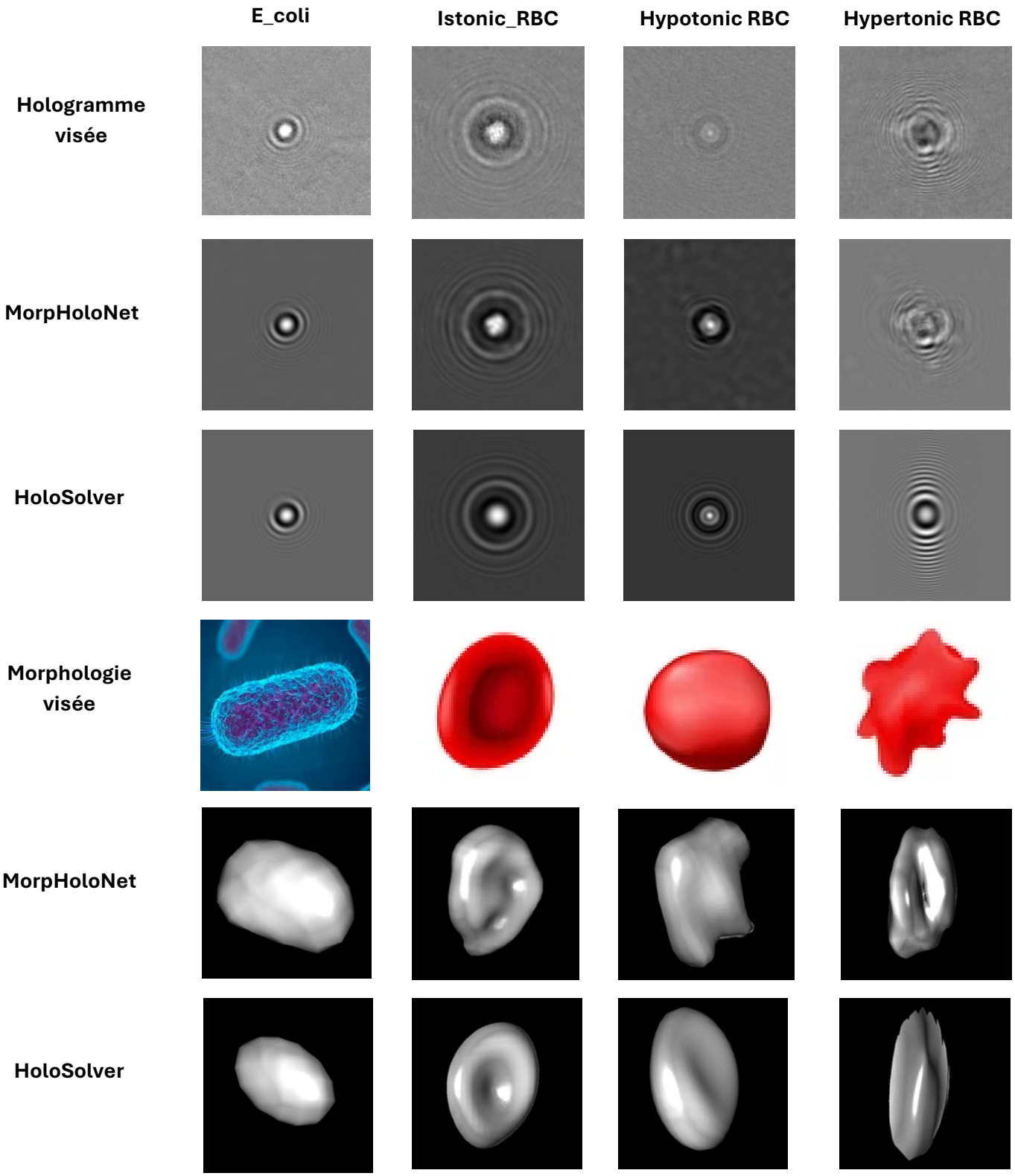
Étant donné la nature mal posée du problème et l'absence de critère d'arrêt automatique fiable (*Early Stopping*), les résultats quantitatifs et qualitatifs présentés ont été sélectionnés manuellement et correspondent, pour chaque modèle, à l'époque où la reconstruction morphologique et le placement des bactéries dans l'espace étaient les meilleurs.

Pour éprouver la robustesse de notre architecture, cette démonstration s'articule en deux phases : une première évaluation en configuration optimale (avec initialisation géométrique) et un test de résilience (sans pré-entraînement).

3.1.1 - Évaluation en configuration optimale (Avec Pré-entraînement)

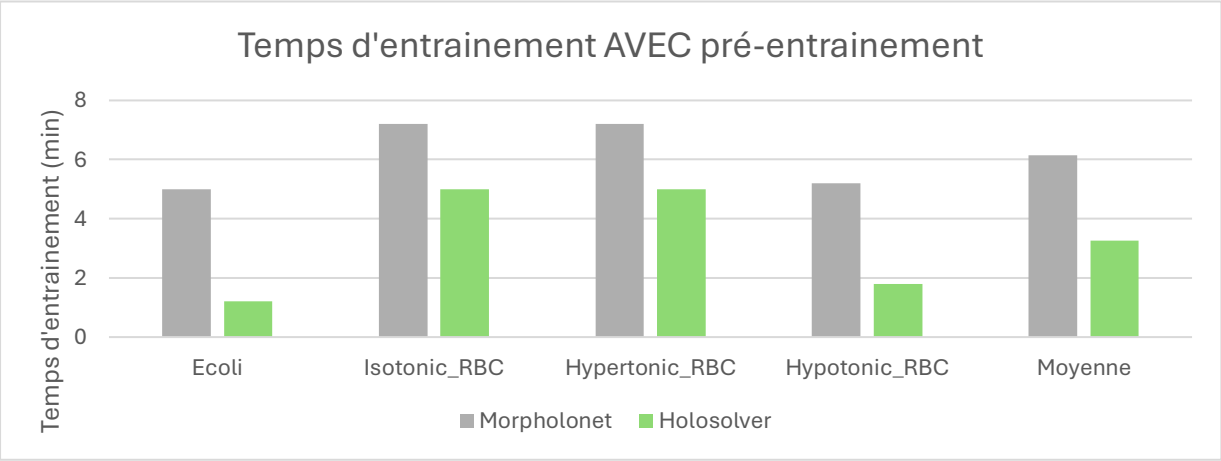
Dans cette première phase, les deux modèles bénéficient d'un pré-entraînement (un *Prior* géométrique basé sur la position approximative de la cible). Il convient de noter que l'implémentation originale de `MorpHoloNet` utilise des matrices de covariance paramétrées manuellement de manière très spécifique pour chaque type de cellule (forçant le réseau à s'initialiser sous forme de cercles parfaits ou d'ellipses très fines selon la cible visée). Bien que cette méthode facilite grandement la tâche du réseau de neurones, elle constitue un biais d'ingénierie fort qui limite la généralisation du modèle à des formes inconnues.

Reconstruction Morphologique et Optique (Analyse Qualitative)



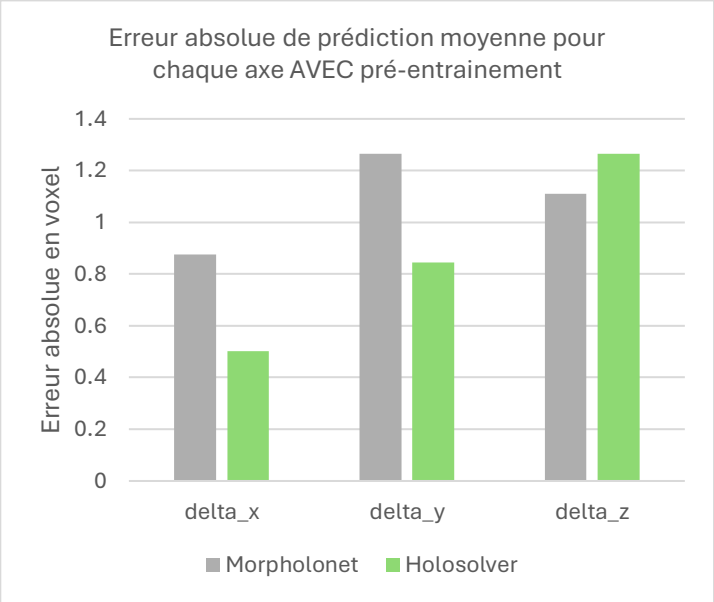
L'analyse visuelle des résultats post-convergence met en évidence des différences notables de modélisation. Sur le plan de l'hologramme reconstruit (simulation 2D), `HoloSolver` génère des motifs de diffraction globalement plus lisses et moins bruités que ceux de `MorpHoloNet`, tout en capturant l'essentiel de l'information de phase. Cette clarté optique se traduit par une supériorité évidente dans la reconstruction 3D. Les volumes sculptés par `HoloSolver` sont exempts d'artefacts haute-fréquence et respectent fidèlement la morphologie visée. L'écart de performance est particulièrement flagrant sur la cible *Hypotonic RBC* (globule rouge gonflé et sphérique) : là où `MorpHoloNet` échoue et produit une forme chaotique et bosselée, `HoloSolver` parvient à reconstruire un volume sphérique lisse et cohérent.

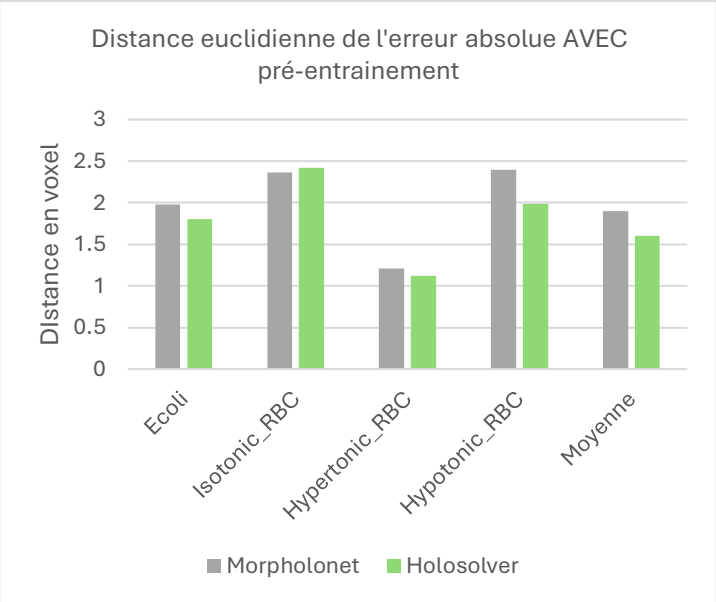
Efficacité Computationnelle



Sur le plan matériel, notre refonte architecturale démontre tout son potentiel, elle permet à `HoloSolver` d'être en moyenne **2 fois plus rapide** que le code TensorFlow d'origine (réduction d'environ 50% du temps d'entraînement). Notre modèle atteint sa meilleure reconstruction en un peu plus de 3 minutes en moyenne, contre plus de 6 minutes pour MorpHoloNet.

Précision Spatiale et Tracking (Analyse Quantitative)





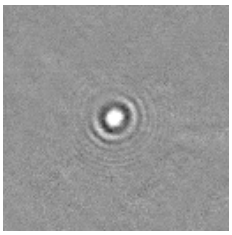
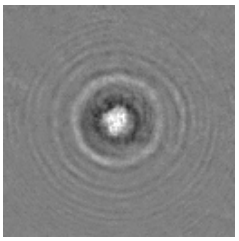
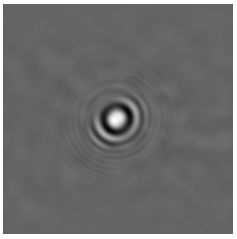
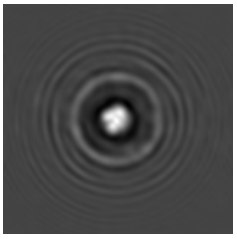
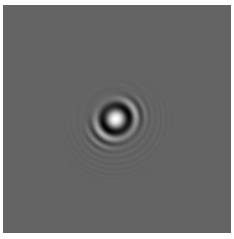
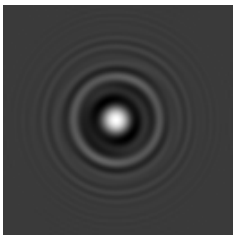
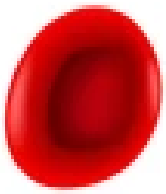
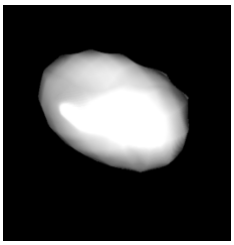
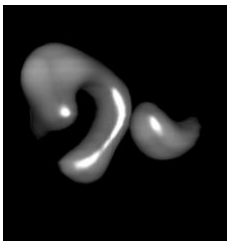
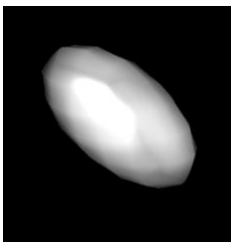
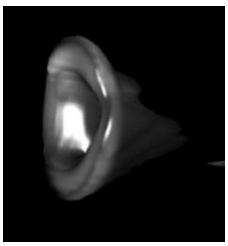
En termes de distance euclidienne globale, `HoloSolver` est en moyenne **15% plus précis** que son prédécesseur. Si `MorpHoloNet` conserve un léger avantage sur la cible *Isotonic RBC*, notre modèle le surpasse sur les autres échantillons.

La décomposition de cette erreur absolue par axe révèle une observation physique fondamentale. L'erreur spatiale de `HoloSolver` est extrêmement faible sur les plans transverses (X et Y) et légèrement supérieure sur l'axe de profondeur (Z). Cette distribution est mathématiquement et optiquement saine, puisqu'elle reflète l'artefact classique d'élongation axiale propre à l'holographie in-line. À l'inverse, `MorpHoloNet` présente une anomalie structurelle : son erreur sur l'axe Y est supérieure à son erreur sur l'axe Z. Ce comportement, contraire aux lois de l'optique ondulatoire, suggère une instabilité dans son encodage spatial que notre nouvelle architecture a su corriger.

3.1.2 - Test de Robustesse (Apprentissage ab initio sans pré-entraînement)

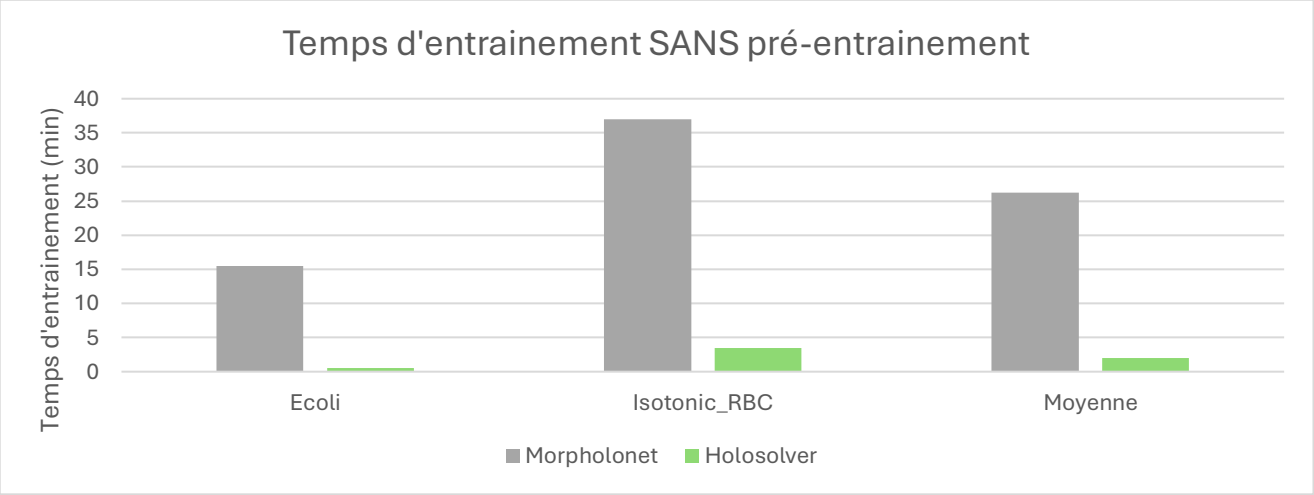
Afin d'évaluer la résilience intrinsèque des deux architectures, une seconde série d'expérimentations a été menée sur deux des échantillons précédents (*E. coli* et *Isotonic RBC*). Dans cette configuration, toute forme de pré-entraînement géométrique a été désactivée. Les modèles doivent donc reconstruire la morphologie et localiser la cible à partir d'une initialisation spatiale purement aléatoire.

Reconstruction Morphologique et Optique (Analyse Qualitative)

	E_coli	Istonic_RBC
Hologramme visée		
MorpHoloNet		
HoloSolver		
Morphologie visée		
MorpHoloNet		
HoloSolver		

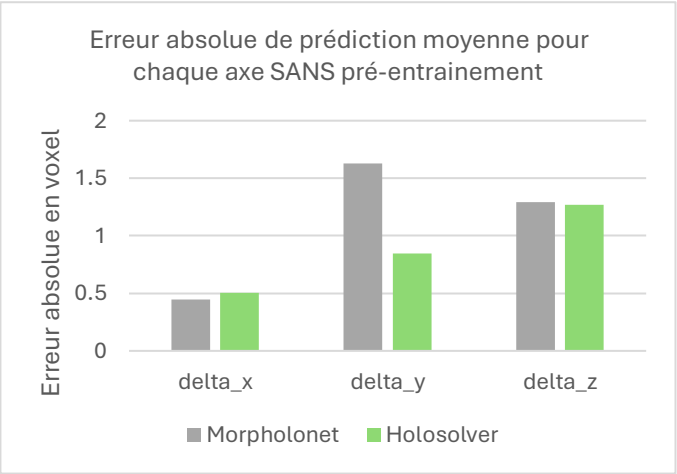
Si la simulation des hologrammes 2D suit la même tendance que lors des essais précédents (un lissage optique supérieur pour `HoloSolver` face au bruit de `MorpHoloNet`), l'inférence des volumes 3D révèle une divergence majeure. Face à la bactérie *E. coli* (morphologie simple), les deux modèles parviennent à converger vers une forme cohérente. En revanche, face à la géométrie biconcave complexe du globule rouge isotonique, l'absence de *Prior* géométrique provoque l'effondrement de `MorpHoloNet`. Ce dernier échoue à modéliser une entité unique et fragmente le volume en de multiples artefacts disjoints ne présentant aucune ressemblance avec la cible. À l'inverse, `HoloSolver` démontre une bonne stabilité. Entraîné depuis zéro, il parvient à retrouver de manière autonome la structure biconcave caractéristique du globule rouge. La seule limite topologique observée réside dans la présence d'un léger artefact d'élongation axiale (traînée sur l'axe Z). Bien que perfectible, cette reconstruction demeure la plus fidèle à la réalité biologique.

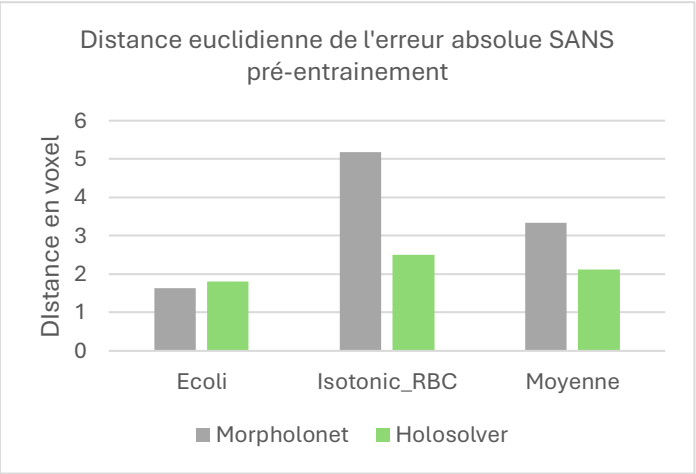
Efficacité Computationnelle



Le test d'apprentissage depuis zéro met en avant l'impact absolu de nos optimisations logicielles et algorithmiques (Section 2.5). Livré à lui-même, `MorpHoloNet` peine à trouver ses gradients et nécessite un temps de convergence rédhibitoire (moyenne de 26 minutes sur ces deux échantillons, avec un pic à plus de 35 minutes pour l'Isotonic RBC). À qualité de reconstruction supérieure, `HoloSolver` converge en moyenne en **2 minutes**, ce qui représente une accélération de **92 %**. La combinaison d'un taux d'apprentissage découplé, d'un paramètre d'inertie calibré à 0.95 et d'un code massivement vectorisé sous PyTorch permet au modèle de traverser les plateaux d'optimisation sans dépendre d'une initialisation avantageuse.

Précision Spatiale et Tracking (Analyse Quantitative)





(Note statistique : Les observations de cette sous-section sont à relativiser, la moyenne n'étant ici calculée que sur un échantillon de taille N=2).

L'analyse suivante confirme la supériorité du suivi spatial de notre architecture. L'erreur absolue de `HoloSolver` par axe demeure quasi identique à celle observée avec pré-entraînement, confirmant que le modèle ne nécessite pas d'indice initial pour localiser précisément le centre de masse de l'objet. L'examen de la distance euclidienne globale scelle cette comparaison. Si `MorpHoloNet` s'effondre littéralement sur la cible isotonique (avec une erreur dépassant les 5 voxels), `HoloSolver` maintient une marge d'erreur contenue autour de 2.5 voxels. En moyenne, notre architecture sans pré-entraînement se révèle **40 % plus précise** que le modèle de référence.

Bilan de la validation comparative

En conclusion de cette phase d'évaluation, `HoloSolver` a fait ses preuves et s'affirme comme bien plus qu'une simple réimplémentation. Il offre des performances supérieures sur trois axes critiques : la fidélité de la reconstruction 3D, la réduction drastique du temps d'entraînement, et la précision spatiale du suivi (*tracking*).

Puisque les deux modèles partagent des fondations mathématiques et des régularisations similaires, cette différence de précision pourrait s'expliquer par la dynamique d'optimisation inhérente à notre ingénierie. La robustesse de notre modèle, capable de localiser la cible de manière totalement autonome sans connaissances *a priori* là où l'architecture de référence s'effondre est le résultat direct du calibrage de nos hyper-paramètres. L'ajustement du taux d'apprentissage (Learning Rate) et la modification de l'inertie de l'optimiseur Adam (β_2)

permettent à `HoloSolver` de naviguer plus efficacement dans l'espace complexe des pertes (minimisation des artefacts), évitant ainsi les minima locaux qui piègent l'ancien modèle.

Toutefois, cette étude a également permis de statuer définitivement sur le rôle de notre stratégie de pré-entraînement. Si le modèle survit sans lui, ce *Prior* géométrique a prouvé son absolu nécessité pour pallier les artefacts optiques d'élongation et sculpter la morphologie de la bactérie avec une exactitude maximale. C'est donc cette architecture complète, alliant la vélocité de notre solveur PyTorch et la robustesse de notre optimisation à la rigueur de notre initialisation, qui sera déployée dans la suite de nos travaux pour relever le défi des environnements complexes.

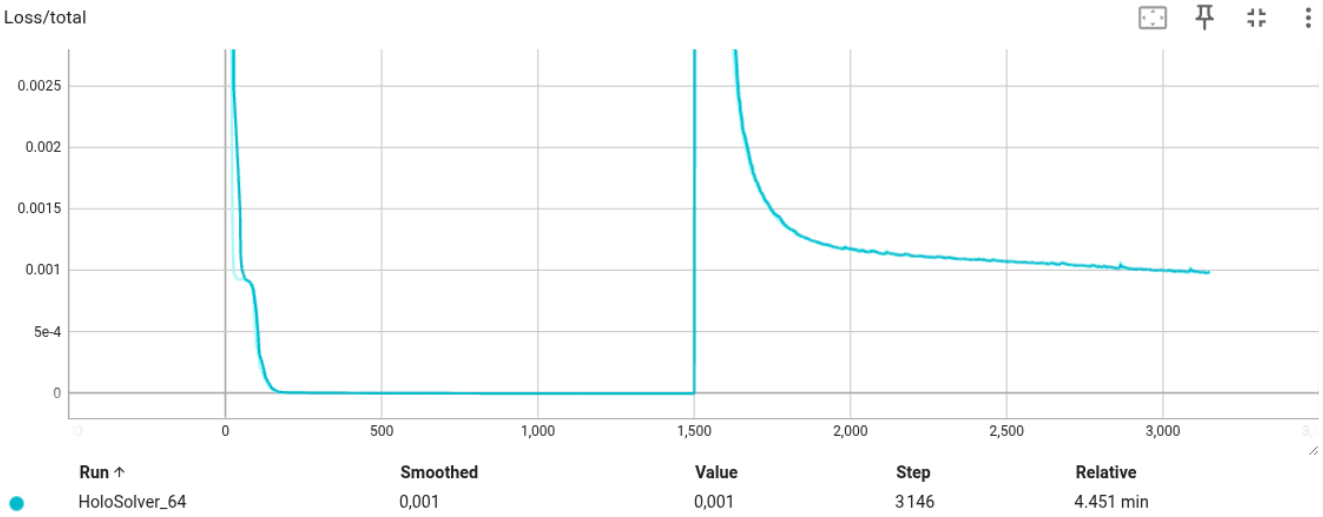
3.2 - Validation sur cibles uniques :

Nous avons principalement travaillé sur des hologrammes de résolution 128×128 pour des raisons de mémoire liées aux GPU dont nous disposions. Dans cette partie, nous allons nous concentrer sur la validation du modèle à l'aide d'hologrammes générés avec une seule bactérie.

Ce choix s'explique d'une part par la difficulté à représenter plusieurs bactéries dans un hologramme de 128×128, en raison de la quantité limitée d'information pouvant y être stockée, et d'autre part parce qu'il s'agit d'un cas plus simple pour analyser nos résultats dans un premier temps.

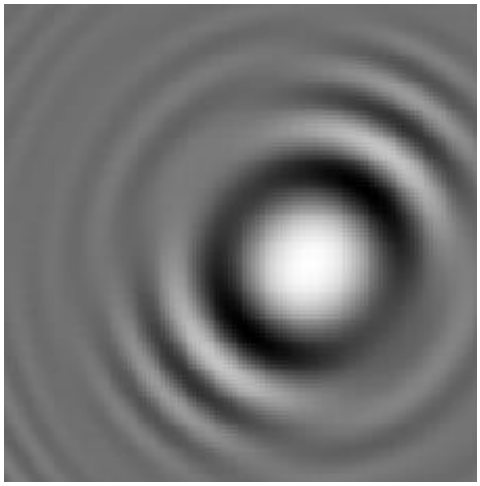
Nous avons réalisé 10 tests sur des hologrammes : 5 d'entre eux sont parfaitement centrés, tandis que l'autre moitié est tronquée et située en bordure d'image. L'objectif est d'évaluer si les performances de détection sont réduites lorsque les bactéries se trouvent en périphérie du champ de vision.

Dans un premier temps, analysons la convergence de la loss à partir de l'exemple ci-dessous, réalisé avec un hologramme de résolution 128 × 128 et centré :

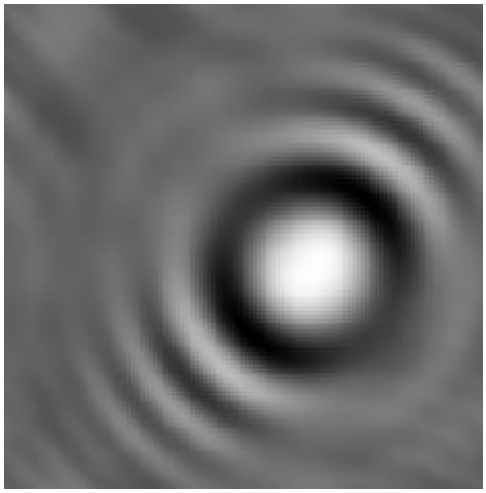


On observe bien la phase de pré-entraînement jusqu'à 1500 époques. La loss converge ensuite très rapidement aux alentours de 2000 epochs (plus ou moins variable selon les exemples). Le temps de calcul est d'environ 2 minutes pour atteindre ce seuil.

Nous constatons également que le modèle arrive à bien reconstituer l'image réelle, comme on peut le voir ci-dessous sur le même exemple avec 3000 epochs :

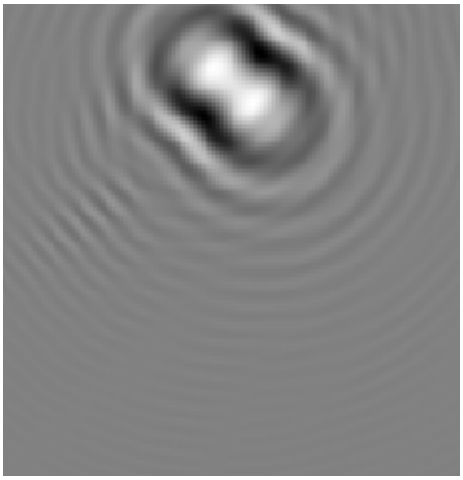


Hologramme réel

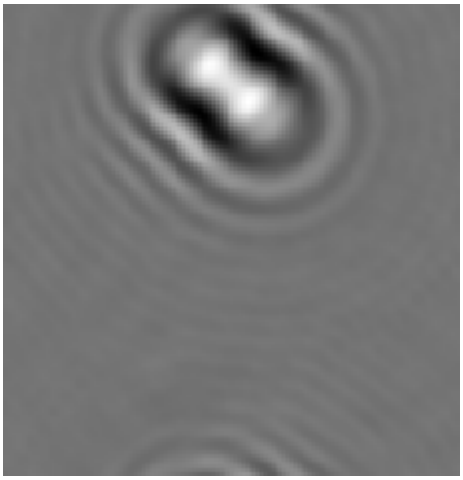


Hologramme reconstitué

Sur cet autre exemple, nous observons que le modèle commence à overfitter au-delà de 4000 epochs, et cela se voit d'autant plus si on laisse le modèle tourner. Concrètement, le modèle commence à interpréter du bruit ou des artefacts comme des données réelles, et on voit apparaître un second hologramme fantôme qui n'existait pas dans les données sources.

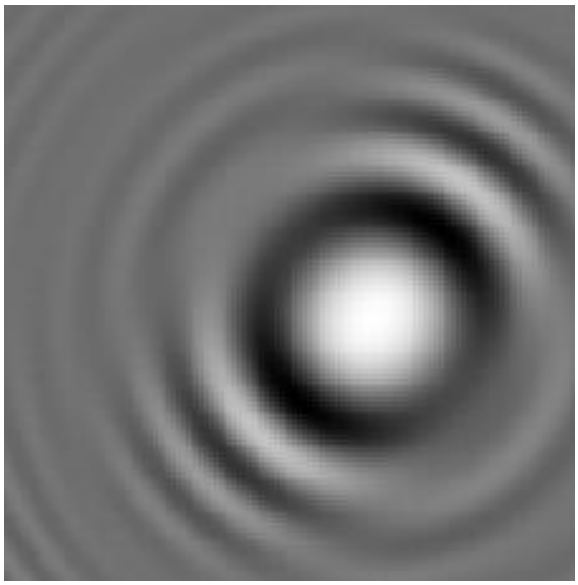


Hologramme réel

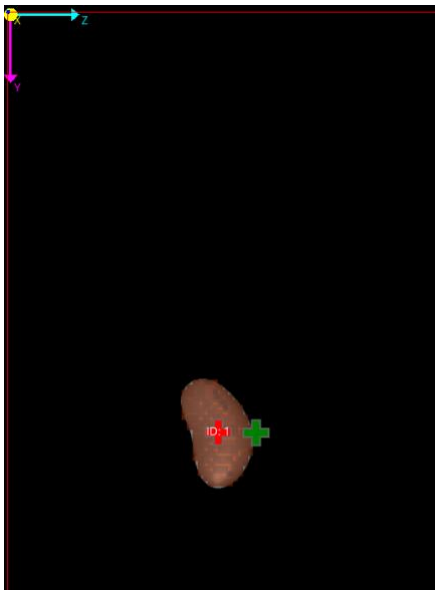


Hologramme reconstitué (2800 epochs)

Avant de passer au tracking, on peut s'arrêter sur la reconstitution 3D de la bactérie. Nous avons obtenu des résultats positifs pour les hologrammes avec une seule bactérie : le modèle parvient à bien la reconstituer. On observe une forme en patatoïde, proche de la forme cylindrique proposée par le générateur. Sur cet exemple, cela est particulièrement visible. D'ailleurs, la croix verte représente la position réelle de la bactérie, tandis que la croix rouge correspond au barycentre de la bactérie reconstituée. On peut ainsi constater que le résultat est très satisfaisant concernant le tracking de la bactérie (Sur cet exemple, l'erreur Δz est d'environ 5 voxels).



Hologramme réel



Reconstitution 3D

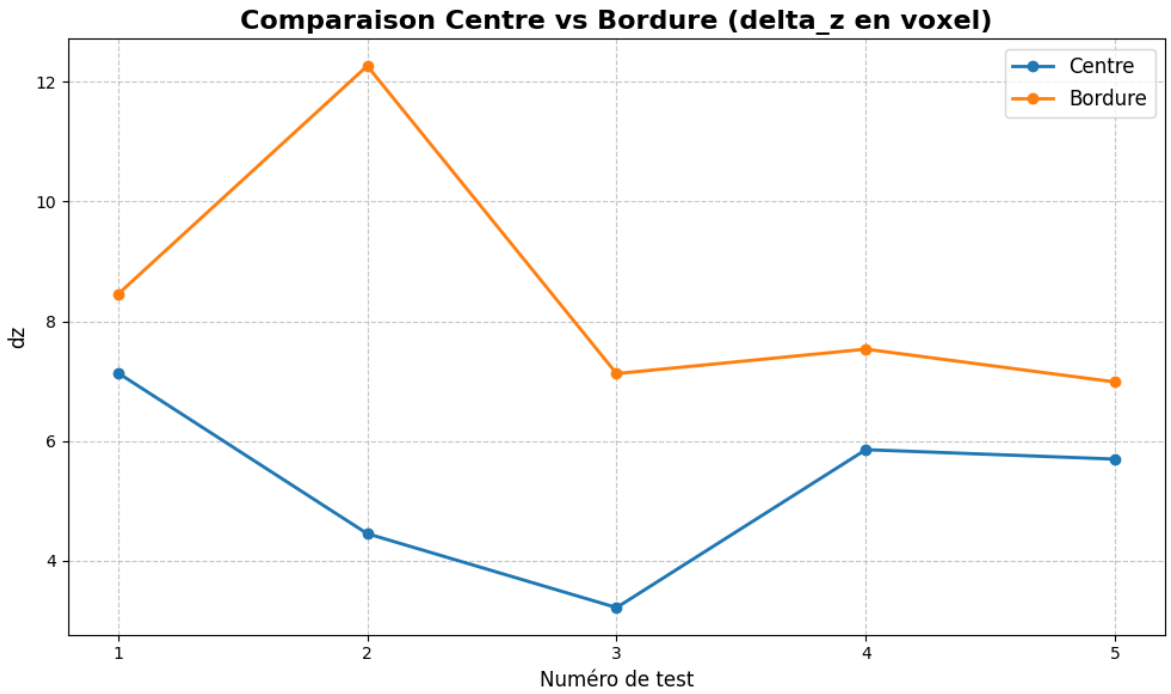
Maintenant, nous allons nous intéresser aux performances en termes de tracking des bactéries. Comme nous l'avons dit plus haut, nous avons pris 10 hologrammes générés à partir d'une seule bactérie. La moitié correspond à des bactéries centrées sur l'image, l'autre moitié étant situées sur les bordures. Nous avons obtenu les résultats suivants :

delta_x (Centre)	delta_y (Centre)	delta_z (Centre)	epochs (Centre)	delta_x (Bordure)	delta_y (Bordure)	delta_z (Bordure)	epochs (Bordure)
0.576	0.576	7.137	3000	0.242	0.082	12.269	3000
0.312	0.154	4.450	3000	0.458	0.421	7.125	3000
0.190	0.298	5.854	3000	1.927	0.763	7.534	3000
0.046	0.120	5.697	3000	0.854	1.055	8.452	3000
0.450	1.102	3.215	3500	1.121	1.896	6.985	3000

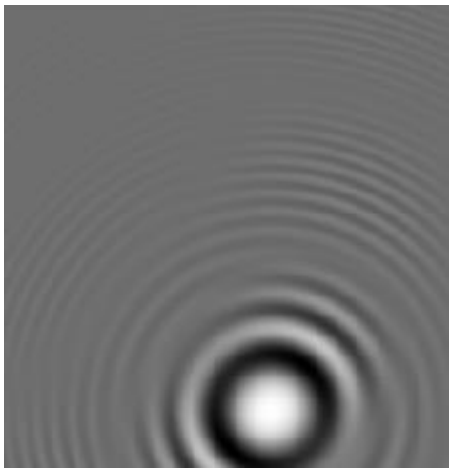
Dans la plupart des cas, on s'arrête à 3000 epochs, car au-delà, le modèle ne converge pas beaucoup plus et a même tendance à overfitter, comme expliqué ci-dessus. Par ailleurs, on remarque qu'en x et y, le delta entre le barycentre trouvé et la vraie position est très faible dans la plupart des essais, ce qui est normal car cette information est simple à extraire à partir de l'hologramme.

Pour z, on obtient des écarts également plutôt faibles, même s'ils sont plus élevés qu'en x et y. On observe que lorsque l'hologramme est bien centré, le delta est plus faible, comme on peut le voir sur le graphique ci-dessous. En revanche, pour les cas où l'hologramme est positionné sur une bordure, le delta est plus élevé. Cela est tout à fait cohérent, car on perd de l'information.

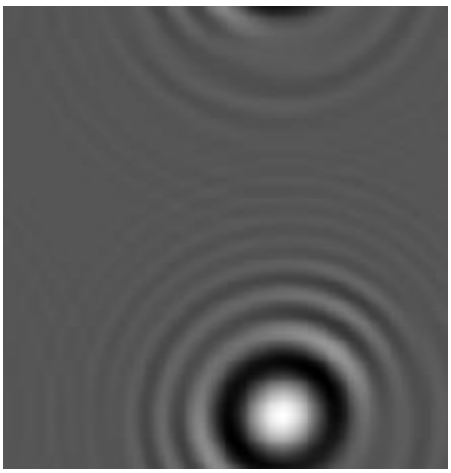
On peut d'ailleurs voir un pic à 12 voxels, qui correspond en fait à un hologramme placé dans un coin et dont on ne voyait presque qu'un quart de la forme.



Enfin nous avons pu faire des tests avec des hologrammes de résolution 256x256 avec une seule bactérie. Là aussi nous avons eu des résultats avec des performances similaires comme avec l'exemple ci-dessous :

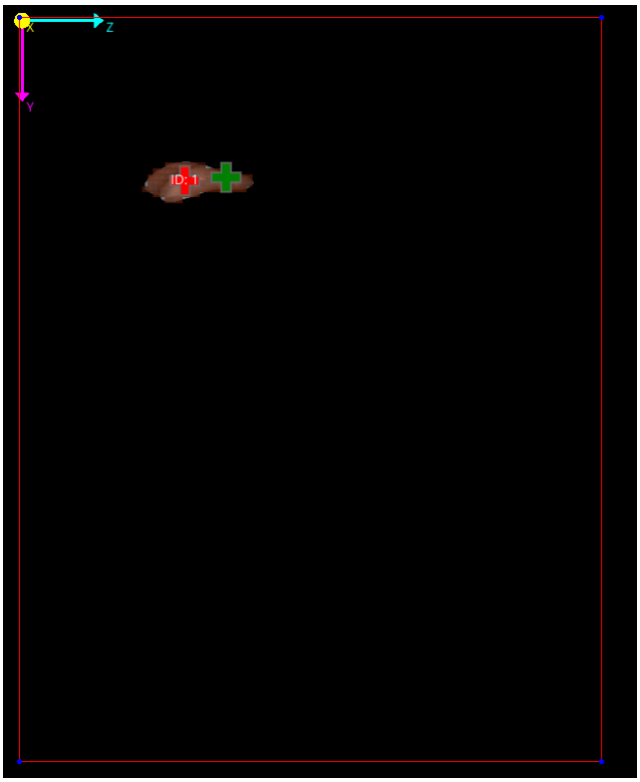


Hologramme réel



Hologramme reconstitué - 3000 epochs

Au niveau de la reconstitution 3D et du tracking, nous avons également eu des résultats convaincants, comme illustré avec le même exemple que ci-dessus :



Reconstitution 3D - 3000 epochs

Delta :

	bacterium_id	x_voxel	y_voxel	z_voxel
0	1	0.629031	0.584899	7.08209

Erreur (en voxel) - 3000 epochs

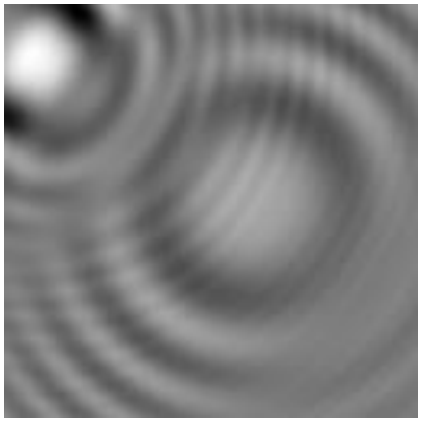
3.3 - Passage à de multiples cibles

Les expérimentations précédentes ayant montré les performances de notre modèle sur des cibles isolées, cette section est dédiée à l'évaluation de `HoloSolver` dans des environnements optiques complexes. L'objectif est double : valider la capacité du modèle à démêler les motifs de diffraction croisés générés par de multiples cibles (interférences destructives et constructives), et éprouver son passage à l'échelle sur des hologrammes haute résolution (de 128x128 à 512x512 pixels). L'ensemble des données de cette section a été synthétisé à l'aide de notre générateur holographique, fournissant une vérité terrain absolue pour les métriques d'évaluation.

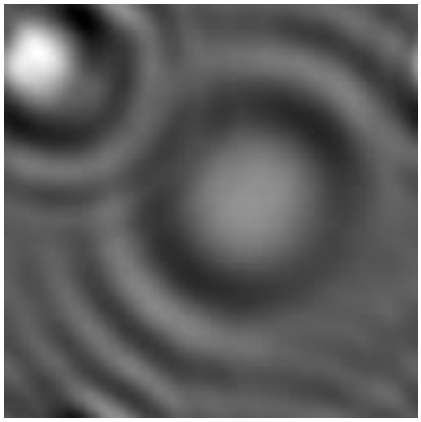
3.3.1 - Preuve de concept et Dynamique d'optimisation (2 cibles, 128x128)

Pour amorcer cette transition vers le multi-cibles, un premier hologramme contenant deux bactéries a été généré dans une résolution standard (128x128). Afin de guider le modèle face à la superposition des franges d'interférence, notre stratégie de pré-entraînement a été activée. La vérité terrain du générateur a servi de connaissance *a priori* pour forcer l'initialisation de deux ellipsoïdes, la forme espérée des cibles étant bacillaire.

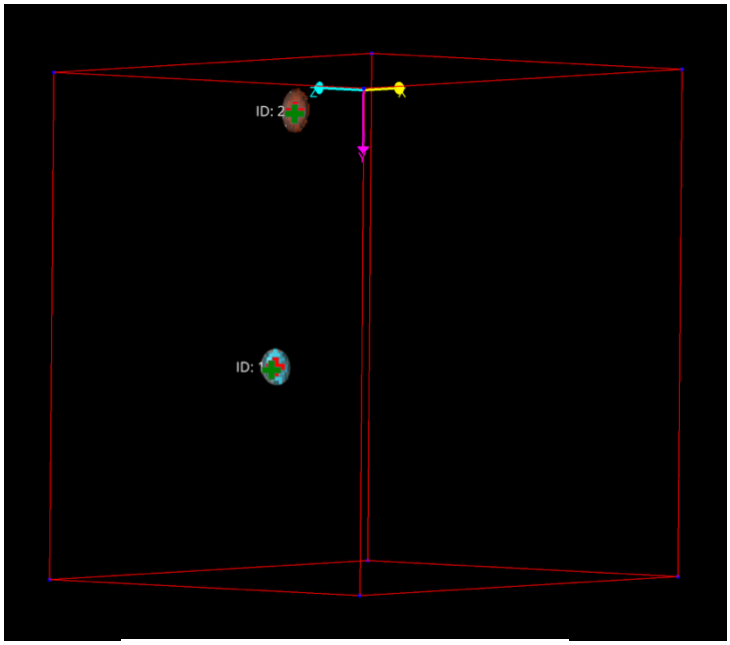
Reconstruction Multi-cibles (Analyse Qualitative)



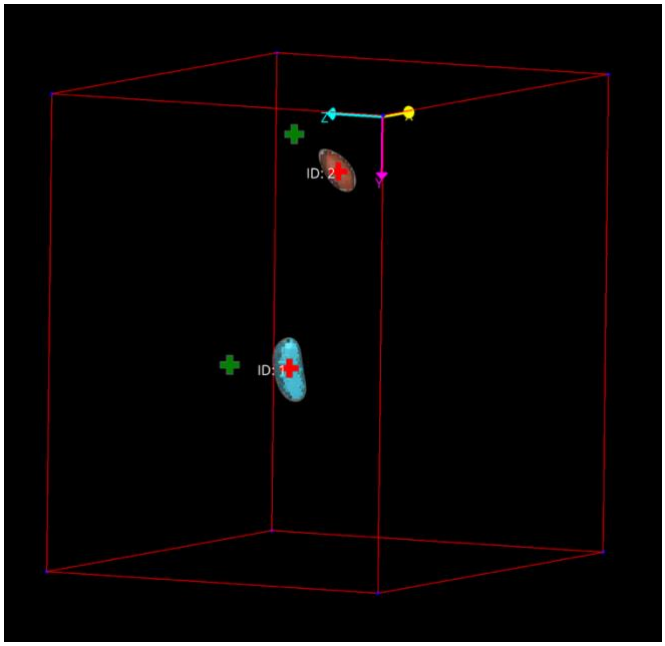
Hologramme visée



Hologramme reconstruit à l'epoch 1000



Reconstruction après pré-entraînement (300 epochs)

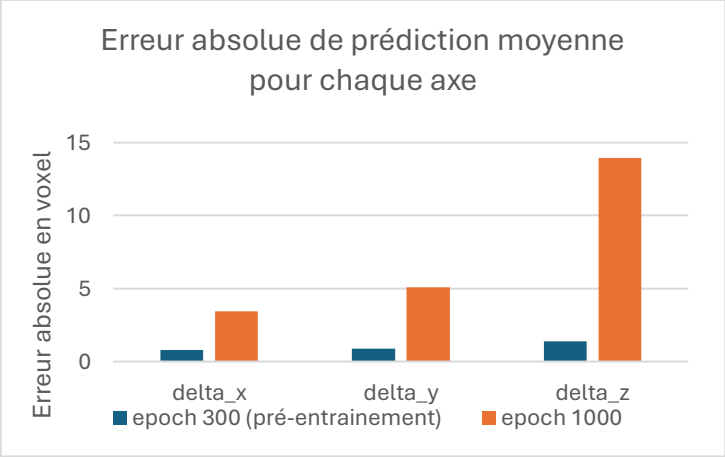


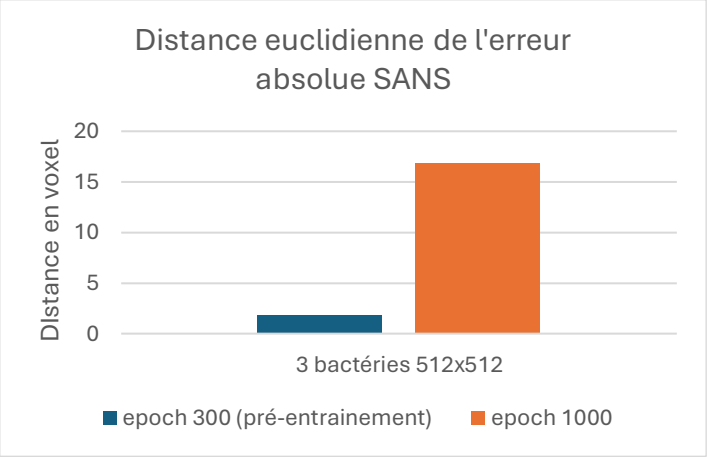
Reconstruction après entraînement (1000 epochs)

L'observation des volumes 3D au cours de l'entraînement révèle la dynamique d'apprentissage du réseau. À l'issue de la phase d'initialisation purement géométrique (Époque 300), le modèle parvient à placer la matière de manière extrêmement proche de la vérité terrain : les centres de masse des deux formes générées coïncident visuellement avec les marqueurs de position.

700 époques plus tard (Époque 1000), l'optimisation a été pilotée exclusivement par le solveur physique. Sur le plan qualitatif, la morphologie est une réussite : le NeRF a parfaitement affiné la densité pour sculpter les formes bacillaires attendues, avec des contours nets et distincts. Néanmoins, on observe visuellement que cette amélioration de la forme s'accompagne d'un décalage spatial. Le modèle semble insister pour modifier la position apprise lors du pré-entraînement, "poussant" les bactéries plus en avant sur l'axe optique par rapport à leur position d'origine.

Précision Spatiale et Tracking (Analyse Quantitative)





L'analyse des métriques confirme cette observation visuelle et met en évidence une forte dérive de la prédiction. Alors que la distance euclidienne moyenne était presque nulle en fin de pré-entraînement (moins de 2 voxels), elle explose à l'époque 1000 pour atteindre près de 17 voxels.

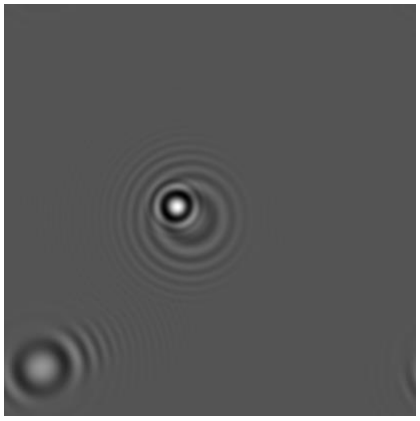
Le graphique de décomposition par axe permet d'identifier l'origine exacte de ce biais. L'erreur est en réalité très faible sur les axes transverses X et Y (moins de 5 voxels), mais culmine à près de 15 voxels en moyenne sur l'axe Z.

Ce phénomène illustre un comportement fondamental en holographie : la compétition entre la contrainte géométrique et la contrainte optique. Pour satisfaire mathématiquement le motif de diffraction croisé sur le capteur 2D et minimiser sa fonction de perte, le solveur physique a tendance à outrepasser la position géométrique initiale. Il déplace la matière sur l'axe de profondeur, subissant l'ambiguïté axiale inhérente à l'holographie *in-line*. Le NeRF ignore donc partiellement notre connaissance *a priori* sur le placement pour privilégier la formation d'une morphologie optiquement cohérente.

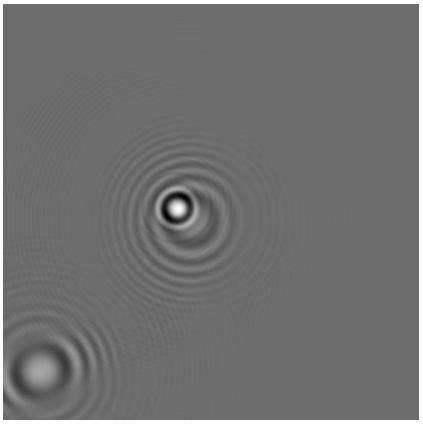
3.3.2 - Le défi de la Haute Résolution et Stabilité Spatiale (3 cibles, 512x512)

Afin d'évaluer la résilience de notre modèle sur des cas d'usage plus proches des standards expérimentaux actuels, la résolution spatiale a été drastiquement augmentée, passant à des hologrammes de 512x512 pixels. Cet échantillon intègre simultanément trois bactéries. Sur le plan de l'ingénierie logicielle, le traitement de ce volume de données (16 fois supérieur au cas précédent) a nécessité l'activation stricte de nos mécanismes de *Chunking* et de *Checkpointing* (détaillés en Section 2.6) pour contenir la consommation de VRAM et éviter la saturation du GPU.

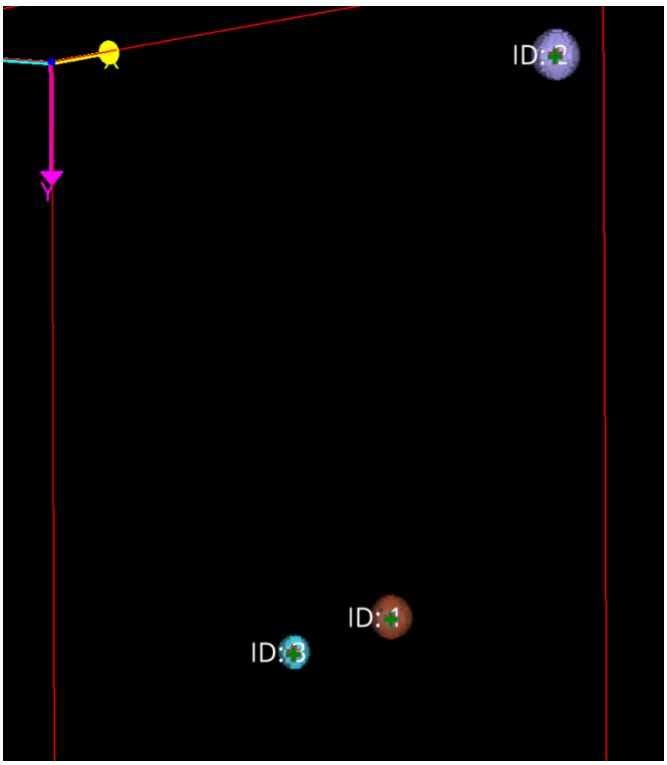
Reconstruction Multi-cibles (Analyse Qualitative)



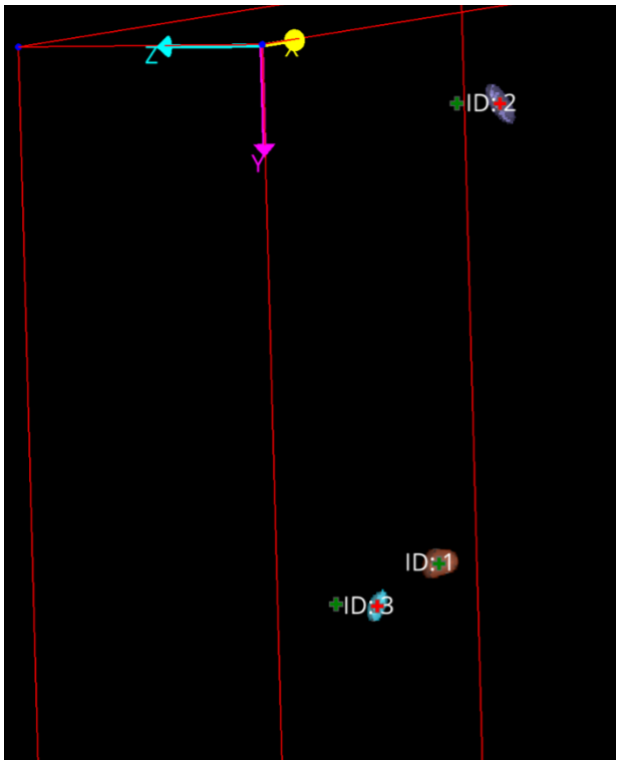
Hologramme visée



Hologramme reconstruit à l'epoch 6000



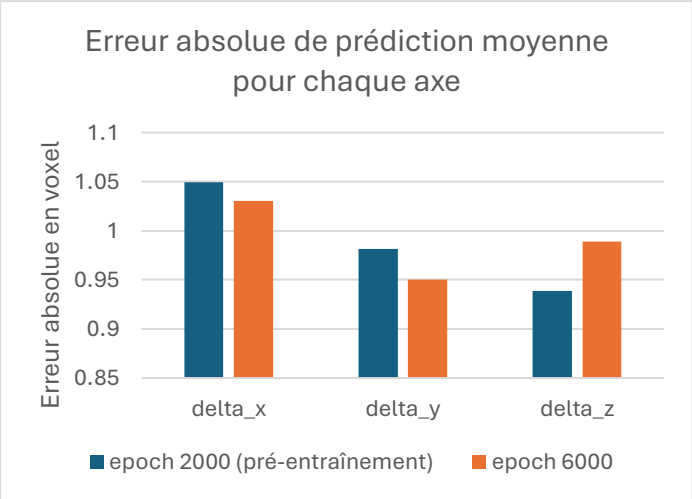
Reconstruction après pré-entraînement (2000 epochs)

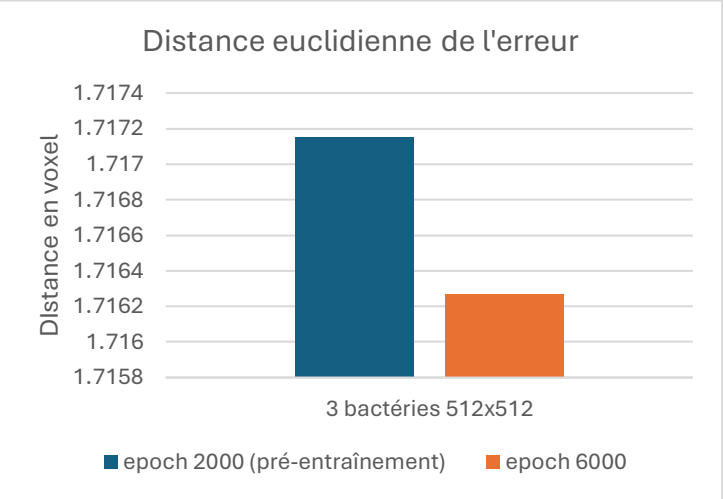


Reconstruction après entraînement (6000 epochs)

L'initialisation géométrique (pré-entraînement) s'est achevée à l'époque 2000, parvenant à placer correctement les trois cibles aux coordonnées espérées. Après 4000 époques d'optimisation purement physique (Époque 6000), le modèle démontre une excellente capacité de séparation : les trois entités sont distinctement détectées, isolées les unes des autres, et leur morphologie bacillaire est préservée sans qu'aucun artefact de fusion n'apparaisse entre elles. De plus, visuellement, la forte dérive spatiale observée sur les images en 128x128 semble ici considérablement atténuée.

Précision Spatiale et Tracking (Analyse Quantitative)





L'examen des métriques de localisation confirme de manière éclatante l'impact de la haute résolution sur le comportement du modèle. Contrairement à l'essai en 128x128 où l'erreur explosait, l'erreur absolue par axe (X, Y, Z) se maintient ici à des niveaux sub-micrométriques, oscillant globalement autour d'un seul voxel.

L'analyse de la distance euclidienne moyenne est la plus révélatrice : celle-ci stagne à environ 1,71 voxel. La variation de cette distance entre la fin du pré-entraînement (époque 2000) et la fin de l'optimisation physique (époque 6000) est de l'ordre du millième de voxel.

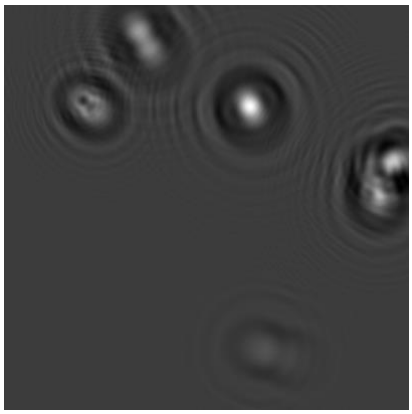
Interprétation : L'apport de la densité d'information

Ces résultats démontrent que l'augmentation de la résolution spatiale stabilise le processus d'optimisation. En 128x128, le manque de pixels à haute fréquence forçait le modèle à chercher un compromis mathématique en déplaçant la matière sur l'axe optique (Z) pour satisfaire le motif de diffraction. En 512x512, la densité d'information optique est suffisante pour contraindre le solveur physique de manière stricte. Le NeRF possède assez de données pour faire converger l'hologramme simulé vers l'hologramme cible, **tout en respectant rigoureusement la connaissance géométrique *a priori***. Le passage à la haute résolution permet ainsi de neutraliser la dérive axiale inhérente au modèle.

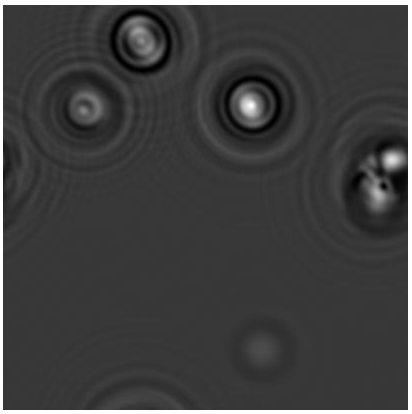
3.3.3 - Limites opérationnelles et Saturation Optique (6 cibles massives, 512x512)

Pour clôturer cette phase d'évaluation, nous avons cherché à atteindre le point de rupture de notre architecture `HoloSolver`. Un ultime hologramme a été généré en 512x512, intégrant cette fois 6 bactéries. Pour maximiser la difficulté, le générateur a été configuré pour produire des bactéries de très grande taille, augmentant ainsi drastiquement la surface et l'intensité de leurs motifs de diffraction respectifs.

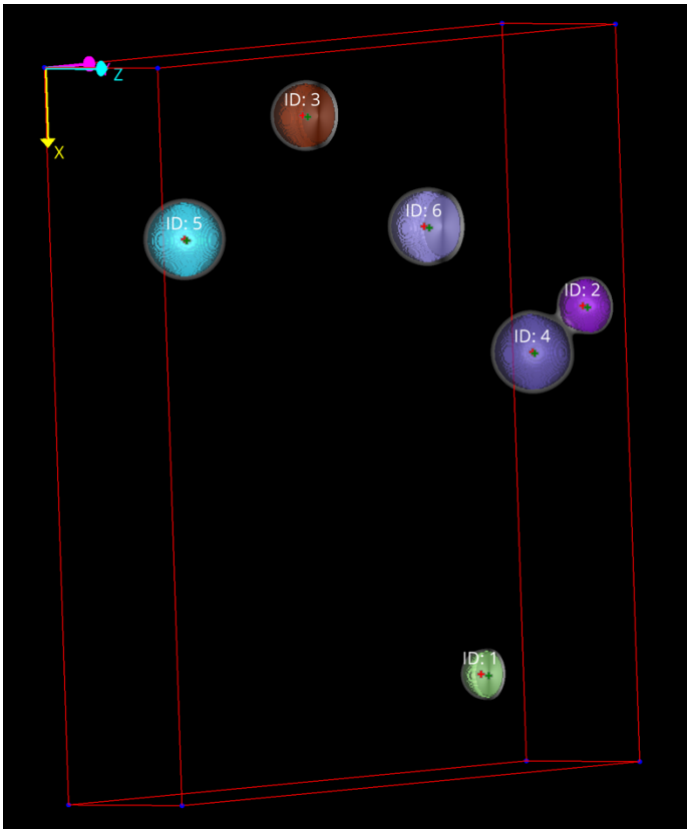
Reconstruction Multi-cibles (Analyse Qualitative)

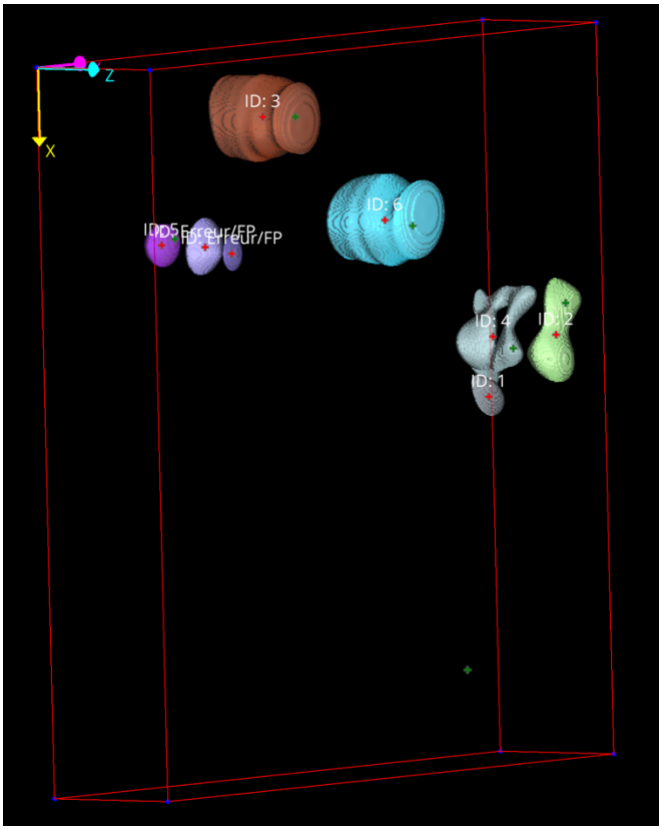


Hologramme visée



Hologramme reconstruit à l'epoch 10000





Si l'initialisation géométrique place correctement toutes les cibles, l'optimisation physique de 10 000 époques aboutit à un échec. Visuellement, le modèle scinde plusieurs bactéries en fragments sur l'axe de profondeur (Z) et fait totalement disparaître la cible la plus excentrée. Parallèlement, la fonction de perte stagne autour de 0.001, confirmant l'incapacité de l'optimiseur à converger.

Interprétation Scientifique

Cet échec illustre les limites optiques et algorithmiques du système face à un problème devenu sévèrement mal posé. La superposition massive des anneaux de diffraction crée un chaos optique qui enferme le réseau dans des minima locaux. Pour reproduire mathématiquement ce motif 2D complexe, le modèle sacrifie la cohérence morphologique 3D en répartissant artificiellement la matière sur plusieurs plans de profondeur (dédoublement axial). La disparition de la cible isolée relève quant à elle d'une famine de gradient (*gradient starvation*) : l'optimiseur concentre toutes ses corrections sur l'erreur massive générée par l'amas central, traitant le faible signal de la bactérie excentrée comme un bruit de fond qui finit par être effacé du volume.

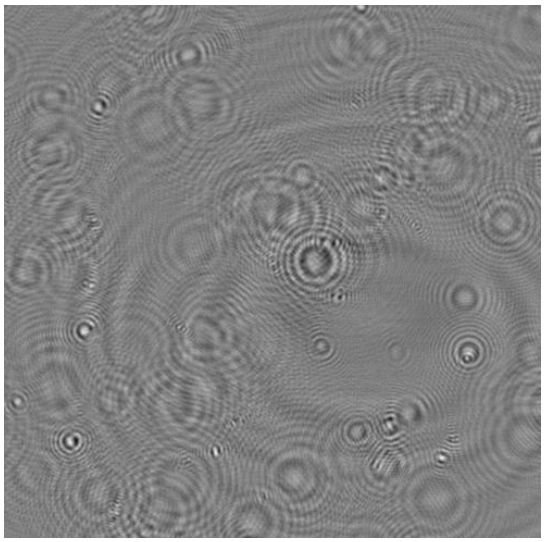
3.3.4 - Bilan du passage à l'échelle (Multi-cibles)

Ces trois expérimentations définissent avec précision l'enveloppe de performance de `HoloSolver` en environnement complexe. Le passage à la haute résolution (512x512) s'est révélé être un levier indispensable : il fournit la densité d'information optique nécessaire pour contraindre le modèle, annuler la dérive axiale observée en basse résolution, et garantir un *tracking* d'une plus grande stabilité.

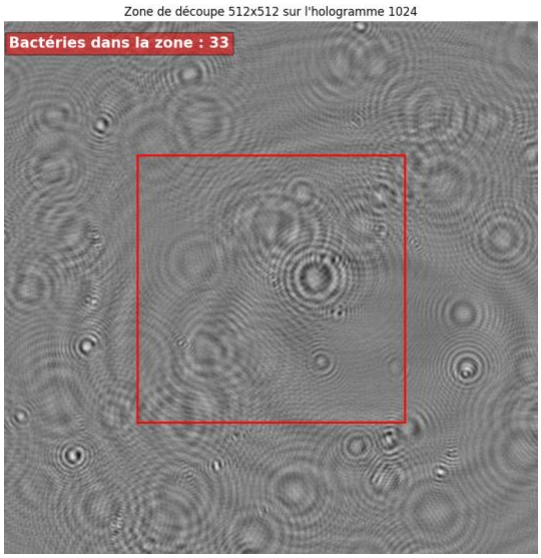
Néanmoins, le test de saturation extrême a mis en lumière la frontière physique du système. Face à un désordre trop dense des anneaux de diffraction, le problème devient sévèrement mal posé, entraînant l'effondrement topologique de la reconstruction.

3.4 - Validation sur un hologramme réel

Nous avons vu dans les parties précédentes que le modèle fonctionnait bien lorsque le nombre d'entités était réduit, mais que les performances commençaient à se dégrader lorsque le nombre de bactéries augmentait dans le volume. Pour tester notre modèle en conditions réelles, nous avons donc demandé à Simon de nous fournir un hologramme réel (voir ci-dessous). Nous avons dû cropper l'hologramme, initialement en 1024x1024, au format 512x512 pour des questions de mémoire, comme expliqué précédemment. Nous avons extrait les bactéries présentes dans la zone et récupéré leurs coordonnées. Nous en avons dénombré 33 (cf. screen).

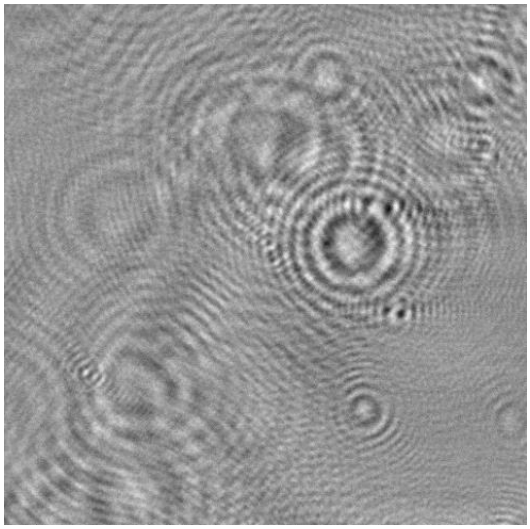


Hologramme réel - 1024

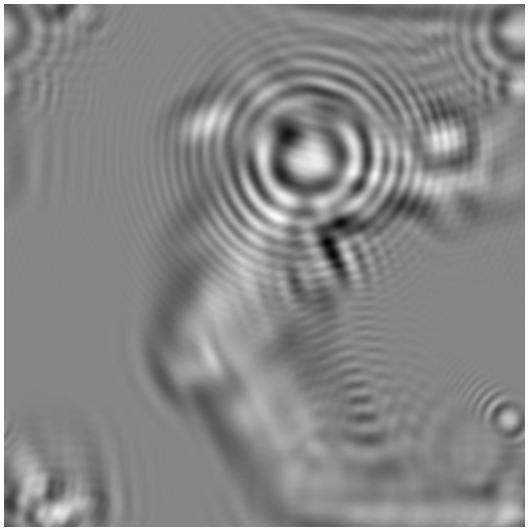


Crop Hologramme réel - 512

Nous n'avons pas eu les résultats espérés. Premièrement, l'hologramme reconstruit (19000 epochs ~ 6h) est fortement bruité et n'a pas détecté toutes les bactéries. Au centre, le modèle semble se forcer à voir un seul hologramme au lieu des multiples hologrammes présents.



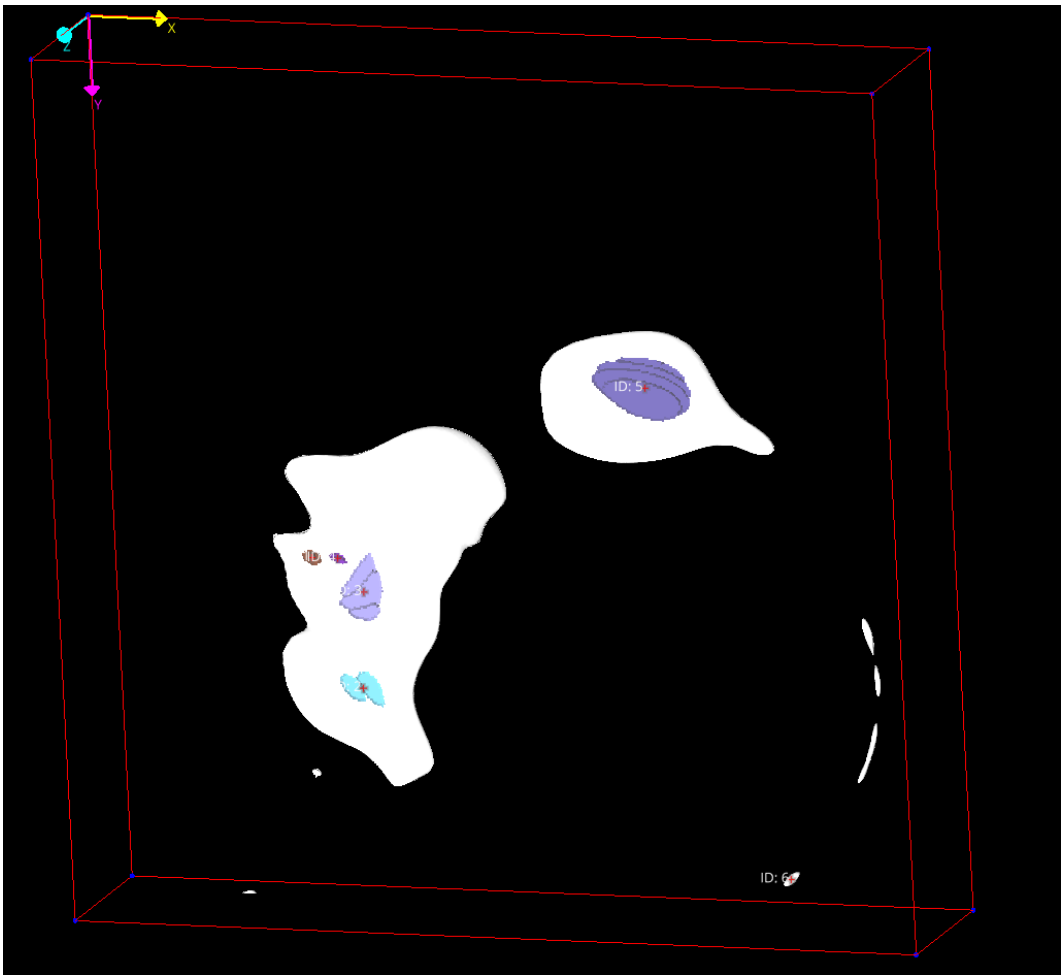
Hologramme réel - 512



Hologramme reconstitué - 512

Un autre problème est que le modèle n'a identifié que 6 bactéries sur les 33 initiales, et le tracking n'est pas satisfaisant. Par ailleurs, la reconstitution n'est pas convaincante, comme on peut le voir ci-dessous : on remarque une "soupe" en plein milieu. Cela correspond au centre de l'image de notre hologramme réel, où de nombreux hologrammes se superposent ; on peut émettre l'hypothèse que le modèle peine à assimiler toutes les informations et ne perçoit à cet endroit qu'un seul hologramme comme on peut le voir dans l'hologramme reconstitué.

Quoi qu'il en soit, cela concorde avec ce que nous pensions : le modèle atteint ses limites lorsque le nombre de bactéries augmente, ce qui entraîne une dégradation des performances. Peut être aurait il fallu désactiver le pré-entraînement également avec autant de bactéries...



Reconstitution 3D

4 – Discussion et Conclusion

4.1 - Interprétation des performances et limites du modèle :

L'évaluation de l'architecture HoloSolver met en évidence une corrélation directe entre la résolution de l'hologramme et la précision de la prédiction spatiale. Sur un format de 128x128 et 256x256 pixels, bien que la reconstruction morphologique soit satisfaisante, le positionnement reste mitigé : l'erreur transversale (X, Y) est faible, mais la dérive axiale (Z) demeure non négligeable, même pour des cibles isolées.

En revanche, l'apport d'information induit par le passage à la haute résolution (512x512) permet de contraindre rigoureusement la physique de diffraction, rendant l'erreur spatiale quasi négligeable et le suivi particulièrement convaincant.

Malgré ces succès, le modèle se heurte à d'importantes limites physiques et matérielles. Dès que la densité dépasse 5 bactéries sur une image en 512x512, les résultats se dégradent. Face à plusieurs dizaines de cibles, le modèle ne parvient plus à distinguer les motifs d'interférence et tend à fusionner les objets 3D entre eux.

De plus, l'approche hybride requiert d'énormes ressources GPU. N'étant capable de traiter nativement que des volumes en 128x128 ou 256x256 sur une carte de 24 Go de VRAM, le passage au 512x512 impose de lourds mécanismes de fragmentation logicielle, étirant le temps de calcul à plusieurs heures et forçant un compromis strict entre la résolution optique et les ressources matérielles.

Dans son état actuel, l'applicabilité réelle du modèle reste donc contrainte. En effet, les biologistes ont précisément besoin d'étudier rapidement des environnements très denses en bactéries, le tout en haute résolution. Dans la pratique expérimentale, un échantillon biologique classique contient rarement quelques cibles isolées, mais plutôt des dizaines de cellules superposées.

De plus, le temps de calcul par image doit être suffisamment court pour permettre de traiter de grands volumes de données en laboratoire. Ainsi, bien que notre travail démontre la viabilité de l'approche NeRF et améliore le modèle de référence, HoloSolver nécessitera de nouvelles étapes de développement et d'optimisation matérielle et logicielle pour franchir le cap et devenir un outil d'analyse déployable en conditions réelles.

4.2 -Difficultés rencontrés :

Pendant le projet, nous avons rencontré plusieurs difficultés. Tout d'abord, l'intégration du générateur d'hologrammes au modèle nous a posé plusieurs problèmes, notamment dans un premier temps pour en comprendre le fonctionnement, puis lors de son intégration au modèle. En effet, il est très important que les paramètres utilisés pour générer les hologrammes correspondent parfaitement à ceux du fichier de config du modèle. Nous avons d'ailleurs obtenu pendant longtemps de mauvais résultats dus à des mauvais paramétrages. Par ailleurs, la compréhension du sens physique et du rôle des différents paramètres nous a également pris du temps.

D'autre part, les ressources GPU disponibles ont été un problème pour nous. Au départ, nous ne pouvions tester notre modèle que sur des hologrammes de taille 128, car les GPU auxquels nous avions accès ne permettaient pas d'aller au-delà, et nous n'avions pas encore effectué toutes les optimisations pour réduire la consommation mémoire.

Une autre difficulté importante est apparue lors de l'implémentation de la grille de hachage (*Hash Grid*), une technique d'encoding plus moderne des NeRF, dont le but était d'accélérer l'entraînement et d'obtenir de meilleurs résultats. Lors de ces tests, nous avons constaté que la reconstruction devenait très instable. Le modèle produisait des artefacts géométriques très marqués, et aucune des régularisations que nous avions implémentées n'a réussi à corriger complètement ce problème.

Enfin, la recherche des meilleurs hyperparamètres pour accélérer le modèle a été longue et difficile. Lors de l'entraînement, nous avons été confrontés à des blocages connus en *Deep Learning*, comme les points de selle (*saddle points*) ou les problèmes de disparition et d'explosion du gradient (*vanishing* et *exploding gradients*). Ces instabilités mathématiques nous ont obligés à revoir à plusieurs reprises nos hyperparamètres (comme les taux d'apprentissage) et le poids de nos régularisations pour réussir à faire converger le modèle

4.3 - Perspectives futures :

Tout d'abord, nous pourrions exploiter pleinement le potentiel des NeRF. En effet, comme vu précédemment dans l'explication des NeRF, plus le nombre de points de vue augmente, plus le modèle est précis. Nous imaginons cependant qu'il est difficile de multiplier les points de vue avec un montage physique réel comme celui du LEMTA. Cela est sûrement très coûteux et en pratique assez complexe à mettre en œuvre. En revanche, on pourrait tout à fait imaginer une amélioration du simulateur. Celui-ci pourrait, à partir d'un volume contenant des positions de bactéries prédéfinies, générer des hologrammes selon différents points de vue. Cela permettrait de réduire considérablement le biais sur l'axe z de notre modèle, d'affiner nos résultats en diminuant le delta d'erreur et de valider l'approche.

Par ailleurs, on pourrait également envisager d'implémenter un tracking avec un suivi dans le temps. En effet, pour l'instant, notre modèle ne permet d'obtenir la position des bactéries qu'à un instant donné. Pour des hologrammes contenant une seule bactérie, le temps d'entraînement est d'environ quelques minutes. En fonction de la précision souhaitée, on pourrait envisager de relancer le modèle à chaque pas de temps Δt tout en utilisant les poids du modèle de l'instant $t - 1$ afin d'obtenir un suivi temporel. Autrement dit avec des pas de temps suffisamment réduits, on pourrait utiliser les positions des bactéries prédites précédemment comme point de départ pour estimer leur nouvelle position.

Il faut toutefois rester vigilant avec les ressources GPU disponibles. Plus Δt sera faible, plus la ressource GPU nécessaire sera importante.

Enfin, le fait que notre modèle ne fonctionne pour l'instant qu'avec un nombre réduit de bactéries est une limite importante, notamment pour une utilisation dans le cadre de projets de recherche au LEMTA. Pour y remédier, une solution serait de découper un hologramme de grande taille (par exemple 1024x1024) en plusieurs sous-hologrammes de taille 128x128. Chacun contiendrait alors moins de bactéries à traiter, et le modèle pourrait être entraîné séparément sur ces différentes zones. Le principal inconvénient de cette approche reste toutefois le besoin en ressources GPU, qui risque d'être fort.

4.4 - Conclusion

En conclusion, nous pouvons dire que Holosolver fonctionne bien dans des situations « simples », avec peu de bactéries et sur de petits hologrammes. En revanche, lorsque l'on augmente la complexité des informations présentes dans l'hologramme, le modèle ne parvient plus à s'y retrouver. Les limites que nous avons atteintes et expliquées précédemment peuvent être améliorées à travers les perspectives proposées dans la partie précédente. Néanmoins, nous avons un doute quant à la pertinence des NeRF dans le cadre de cette problématique précise, à savoir le tracking en temps réel de bactéries dans un volume, en particulier lorsque leur nombre est élevé et que le besoin de précision est important. En effet, si l'application nécessite un suivi précis des bactéries, ainsi qu'un suivi continu (avec un pas de temps le plus faible possible), nous pensons que dans ce type de cas, les NeRF ne sont peut-être pas la solution la plus adaptée.